

Summer-2024

CSE 111

Progamming Language 2
[JAVA OOP]

Made by RAIHAN

Introduction to Java

Why Java?

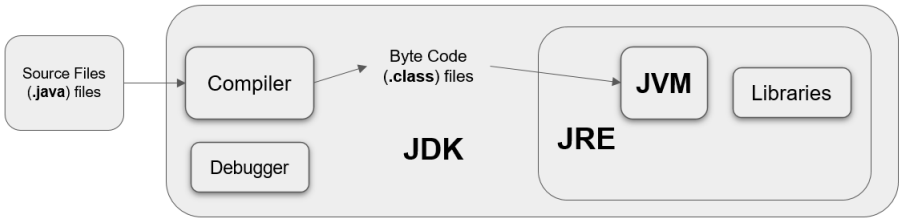
- Runs on more than **5.5 billion devices**. Java is versatile and applicable to Web Development, Mobile Apps (Android), Desktop Applications, IOT Devices and more.
- Java's **OOP** implementation is widely regarded as robust and effective.
- **Enterprise and Large-Scale Systems**. Java is trusted for its stability, scalability, and security in mission-critical projects.
- **Platform Independency** allows Java to compile it's code once and run in any OS with a JVM (Java Virtual Machine).

Platform Independency - A key feature of Java

In order to understand Platform Independency, we must know about **three core components** of Java and understand how Java executes its codes.

- ***JVM (Java Virtual Machine)*** is platform dependent interpreter. This is the component that **executes bytecode**. Mac, Linux, Windows each has different JVMs.
- ***JRE (Java Runtime Environment)*** contains the **environment and libraries** that JVM uses to execute Java programs. JRE includes JVM as well. JRE is required to RUN any java program.
- ***JDK (Java Development Kit)*** contains various tools for developers like **Java Compiler (javac)**, **Java Debugger (jdb)** etc. JDK is required for coding and developing Java applications. JRE is included with JDK.

1) Java code goes through the compiler within the JDK.
2) **Compiler** converts **Java Code to Byte Code** which is **platform independent**. Now this ByteCode can be run in any OS without compiling.
3) This ByteCode (.class file) is executed by the **JVM** using various libraries from the **JRE**.



Installing JDK & DrJava

For this course we'll be using JDK version 8.
Link to JDK version 8 and DrJava IDE:
https://drive.google.com/drive/folders/1pzTIouwS9-tHVEb9HdBnp1WL7NDrK_BI
JDK and DrJava Installation Tutorial:

https://www.youtube.com/watch?v=Gss9sL3Q-8s

Writing Your First Program

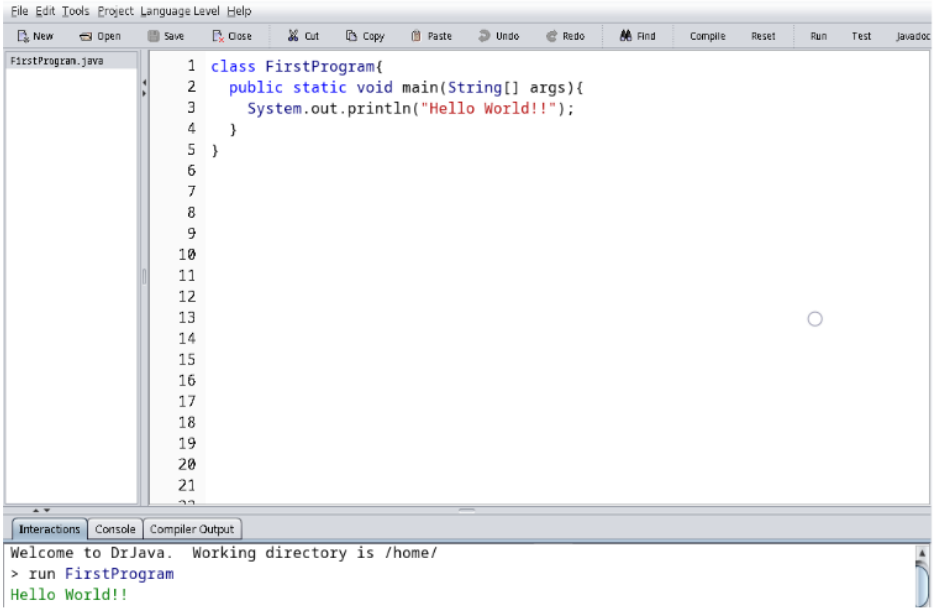
Let's write our first program using Java

```
1.  public class FirstProgram {
2.    public static void main (String [] args){
3.       System.out.print("Hello World");
4.    }
5.  }
```

```
Welcome to DrJava.  Working directory is /home/Documents
> run FirstProgram
Hello World!!
```

This program will print out "Hello World".

Running Java code using DrJava



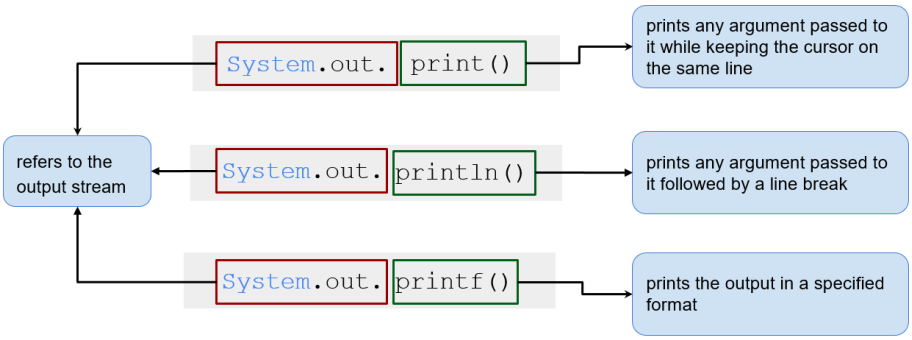
After writing our code using Dr Java's editor, we have to

1. Save the code.
2. Compile the code.
3. Run the code.

Keep in mind, you file name has to be the same as the **Class name.**
In this example, **FirstProgram** is our class name.

Printing in Java

To show the output in the console, java uses three different approaches.




```
1.  public class PrintingExample{
2.       public static void main(String[] args)  {
3.         float y = 1.56f;
4.         System.out.print("Number: " +  23);
5.         System.out.println("Hello World");
6.         System.out.printf("Formatted y = %.4f\n", y);
7.         }
8.  }
```

```
Welcome to DrJava. Working directory is C:\Users\Desktop
> run PrintingExample
Number: 23Hello World
Formatted y = 1.5600
```

অর্থাৎ, System.out.print() এর ক্ষেত্রে পাশাপাশি print করবে। System.out.println() এর ক্ষেত্রে পরের লাইনে print করবে। Cursor Next Line এ চলে যাবে।

Comments in Java

- Comments are used to make the code more readable and easily understandable.
- The commented portions of code are ignored by the java compiler.
- **Single line comments:** Two forward slashes (//) are used at the beginning of the line.
- **Multi-line comments:** It starts with /* and ends with */

```
1.  public class Comments{
2.        public static void main(String[] args)  {
3.          // This is an example of single-line comment
4.          System.out.println("Comments in Java");
5.          /* This is an example of
6.           multi-line comments */
7.          System.out.println("Program ends!");
8.          }
9.  }
```

```
Welcome to DrJava. Working directory is C:\Users\Desktop
> run Comments
Comments in Java
Program ends!
```

Java Keywords

Java Keywords are few reserved words that have predefined meanings and can not be used as variable names. Let's get to know some of them.

1. **public**: An access modifier keyword. It means the class/method is accessible from any other classes. Private and Protected are other types of access modifiers.

◆ public → access modifier. [open to all]

2. **class**: keyword is used to specify the classname. In our example, class FirstProgram means that FirstProgram is the classname.

◆ class → ১ টা Blue Print/Map/Design. এই Design কে ব্যবহার করে Multiple instance বা বিভিন্ন উদাহরণ তৈরি করা হলো object.

```
1.  public class FirstProgram {
2.    public static void main (String [] args){
3.      System.out.print("Hello World");
4.    }
5.  }
```

3. **void**: This keyword means nothing will be returned from the method. We'll learn more about these in the later lectures.
4. **static**: It means the method can be invoked without creating an instance of the class. Again, it'll all make sense when we learn about these in the upcoming lectures.

◆ static → class property / No need to use any object.

◆ main → Name of the method.

```
1.  public class FirstProgram {
2.    public static void main (String [] args){
3.      System.out.print("Hello World");
4.    }
5.  }
```

Java Scopes

In Java, scopes define the visibility and lifetime of variables, methods, and classes. Primarily these scopes are defined using **{ } (Curly Braces)**.

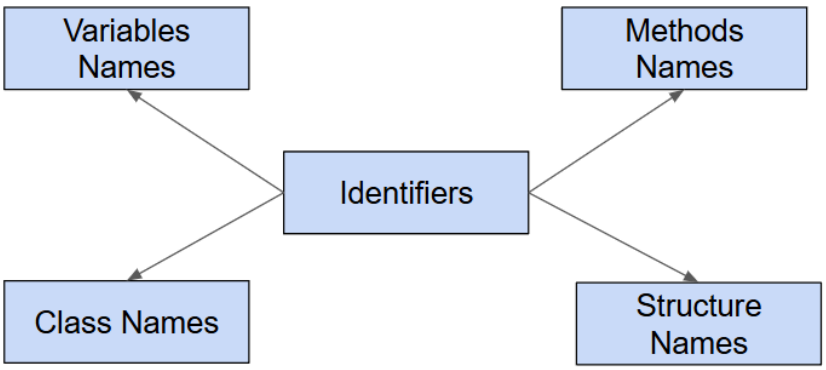
As shown in the example, after declaring the classname “FirstProgram” in the line 1, **“{“** was used. It meant everything below was within **Class Scope** until the bracket was closed in line 5 using **“}”**.

```
1.  public class FirstProgram {
2.      public static void main (String [] args) {
3.          System.out.print("Hello World");
4.      }
5.  }
```

Introduction to
Variables and Datatypes

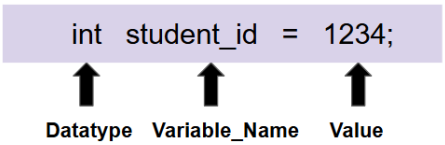
Identifiers

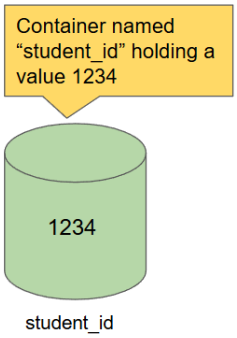
- Name assigned to a particular entity in a program.
- Can be a variable name, method name, class name, structure name etc.



Variables

- A variable is a container which is used to store data.
- A kind of identifier.
- Each variable has a specific data type.





Variables Initialization

- **Syntax**:

<datatype> <variable_name> = <value> <format_specifiers>
[Format specifiers only for double and float data type]

- **Example**:

```
int  number = 25;
char var = 'H';
double value = 2.56d;
```

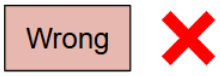
Variable Naming Conventions

- Must begin with letters (A-Z, a-z), dollar sign ($) or underscore (_).

Ex: int number = 90;

- Can not start with numbers or special characters.

Ex: int %name = 88;
Ex: int 1name = 70;



- Can have numbers except at the start (0-9).

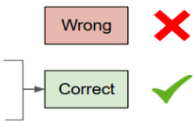
Ex: int num1 = 10;

- Case sensitive

Num1 and num1 not same

- Can not contain white spaces
- Can use **camel casing** (studentName) or **snake casing** (student_id) for multiple words in a variable

Ex: int given number = 20;
Ex: int given_number = 20;
Ex: int givenNumber = 20;



Valid		Invalid	
a123bc	abc123	123Abc	ab cd
first_name	firstName	first name	Ab.cd
Int	$ami	#number	if
_ami	class_1	class	int

- Should be meaningful.
- Single character variable names should be avoided.
- Variable names must not be a Java reserved keyword.

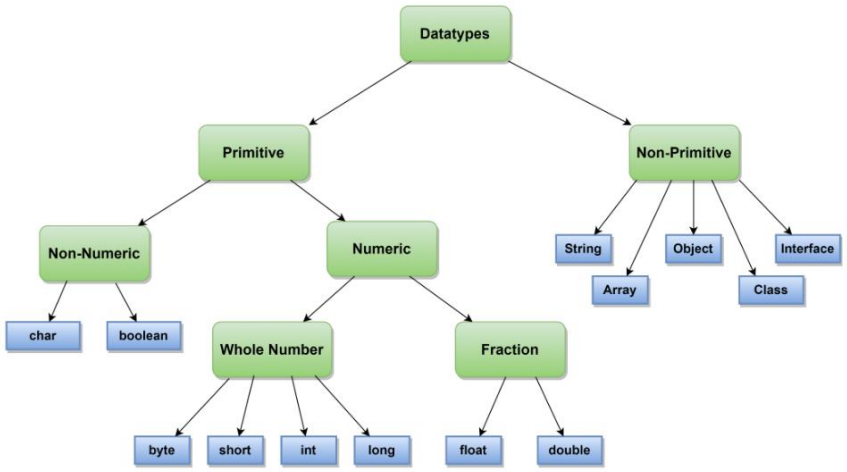
abstract	assert	boolean	synchronized	byte	case	catch	char	class
continue	default	do	double	else	enum	extends	final	finally
float	for	if	implements	import	int	long	native	new
null	package	private	instanceOf	public	return	short	static	strictfp
super	switch	break	interface	this	throw	throws	transient	try
void	volatile	while	protected	const	goto	true	false	null

Fig: Reserved keywords in Java

Data Types

- Each variables in Java must of a specific data type.
- There are major two types of data in Java:

1. **Primitive Data Types:** Holds a value, specifies the size and type of a variable.
2. **Non-Primitive Data Types:** Holds a reference to an object or location.



Data Type	Significance	Range
byte	8-bit integer	-128 to 127
short	Short integer	-32,768 to 32,767
int	Integer value	–2,147,483,648 to 2,147,483,647
long	Long integer	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	Single-precision floating point	1.4e-45 to 3.4e+38
double	Double-precision floating point	4.9e-324 to 1.8e+308
boolean	True/ False value	-
char	Character	-

Primitive Data Types Example:

```
byte value = 20;
short s1 = 1000;
long var = 2351678;
int a = 10;
float b = 1.56f;              // f specifies float value
double d1 = 12.6543d;         // d specifies double value
char ch = 'a';                // Enclosed by single quotation
boolean answer = true;
```

Non-primitive Data Types Example:

- **String**: A sequence of characters surrounded by **double quotes.**
    - **Example**:

String my_string = "Hello World";

- **Arrays**: Collection of a fixed number of values of similar data type.
    - **Example:**

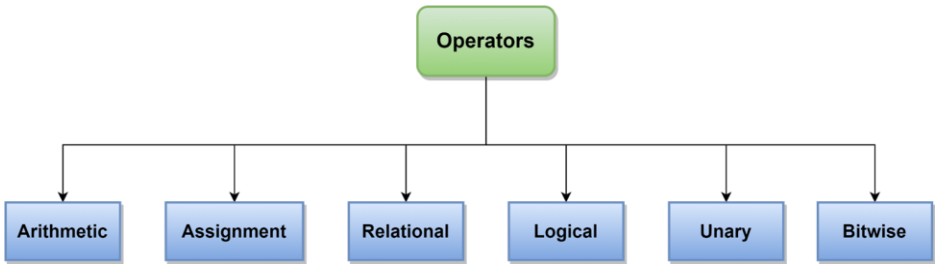
int [ ] arr1 = {24, 9, -11, 5};

PRIMITIVE VS NON-PRIMITIVE

Predefined, no need to create before using	Must be created (except for String) before using
Contains a value	Contains a reference
Cannot be null	Can be null
The size of a primitive type depends on the data type	All non-primitive types have the same size.

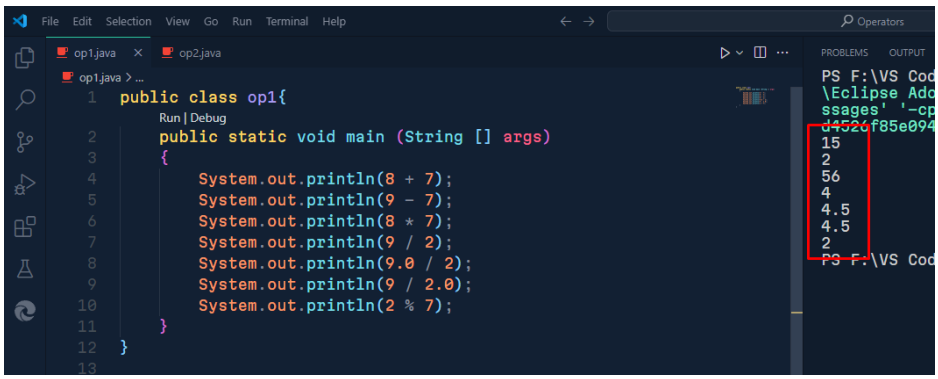
Introduction to Operators

- Operators are symbols used to perform specific operations on one or more operands to produce a result.
- Types of operators in Java:



Arithmetic Operators:

Category	Name	Symbol
Additive	Addition	+
	Subtraction	-
Multiplicative	Multiplication	*
	Division	/
	Modulus	%



Modulus (%): ⊠ Remainder (ভাগশেষ)

2%3 → 3) 2 (0
0
2

ছোট সংখ্যাকে বড় সংখ্যা দ্বারা ভাগ করলে ভাগশেষ হবে ছোট সংখ্যা।

Assignment Operators:

Name	Symbol	Identicality
Assignment	=	a = 5;
Addition Equals	+=	a += 3; a = a+3;
Subtraction Equals	-=	a -= 3; a = a-3;
Multiplication Equals	*=	a *= 2; a = a*2;
Division Equals	/=	a /= 2; a = a/2;
Modulus Equals	%=	a %= 2; a = a%2;



Relational Operators:

Category	Name	Symbol
Comparison	Greater Than	>
	Less Than	<
	Greater Than or Equals	>=
	Less Than or Equals	<=
Equality	Equals	==
	Not Equals	!=



এক্ষেত্রে Boolean Value দেয়।

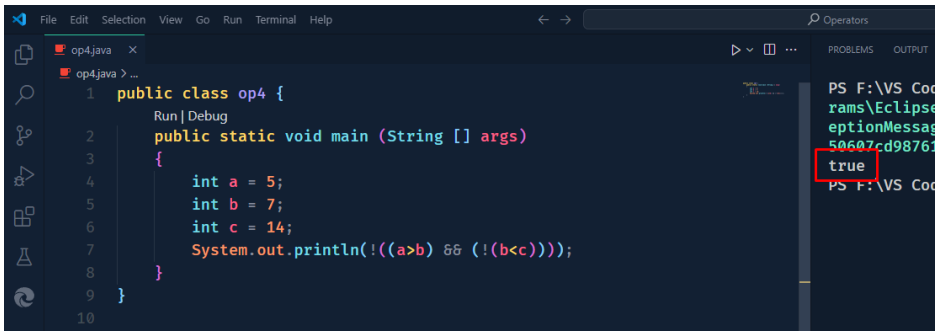
Logical Operators:

Name	Symbol
Logical AND	&&
Logical OR	||
Logical NOT	!

AND (&&): Denotes true if both the conditions before and after are true, otherwise false.

Example:





A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

◆ অর্থাৎ, ২ টি শর্তই true হতে হবে নয়তো output আসবে false.

OR (||): Denotes true if at least one conditions before and after are true, otherwise false.

Example:



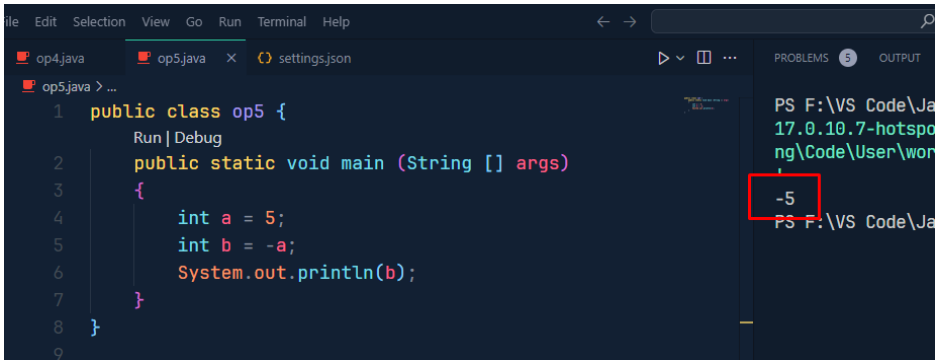
A	B	A || B
true	true	true
true	false	true
false	true	true
false	false	false

- অর্থাৎ, যেকোনো ১ টি শর্ত true হলেই output আসবে true.

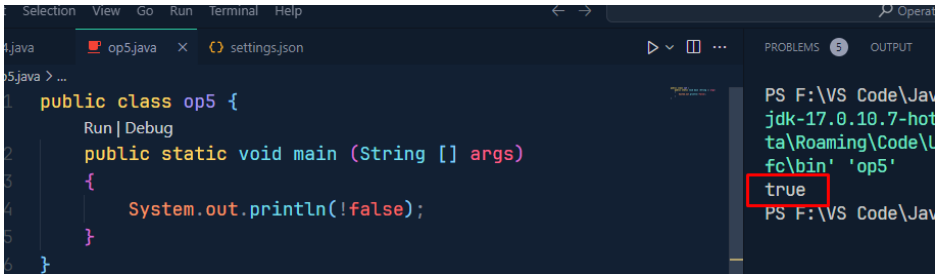
Unary Operators:

Type	Name	Symbol
Prefix	Negation	-
	Not	!
	Increment	++
	Decrement	--
Postfix	Increment	++
	Decrement	--

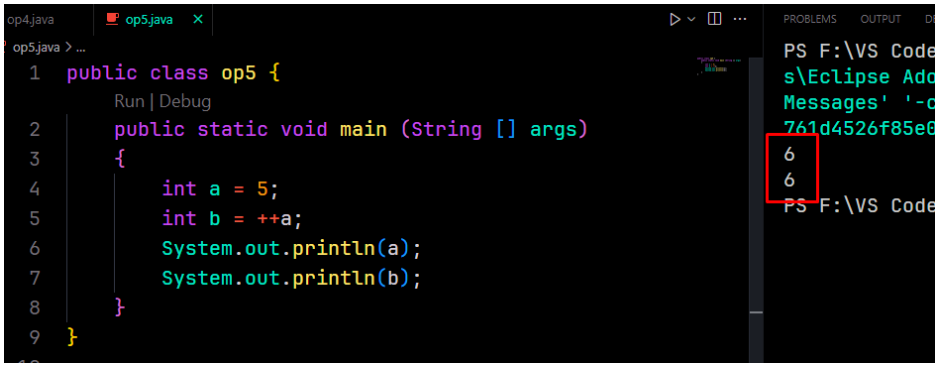
Negation (-): Negates an operand.



Not (!): Reverses the logical value of an operand.

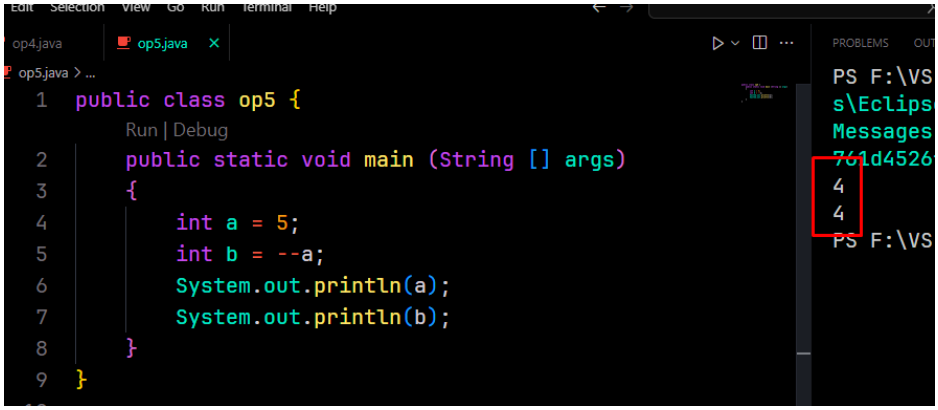


Pre-Increment (++): Increments the operand by 1 and then assigned.



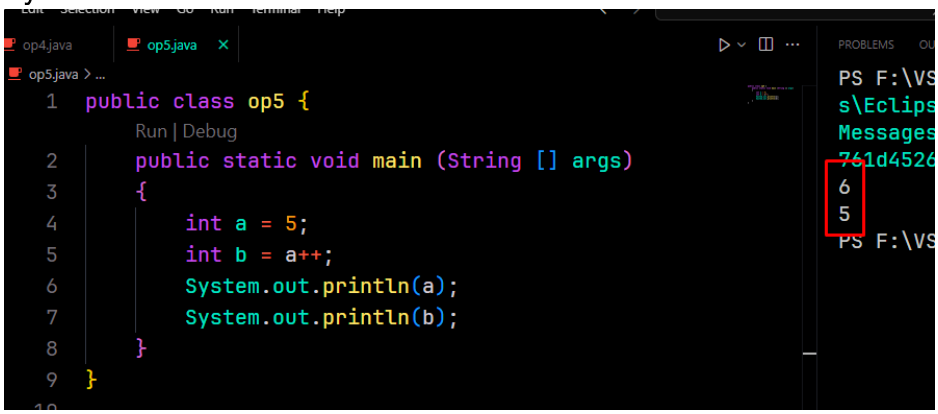
এখানে, a এর value আগে ১ বাড়বে, তারপর সেটা b তে বসে যাবে।

Pre-Decrement (--): Decrements the operand by 1 and then assigned.



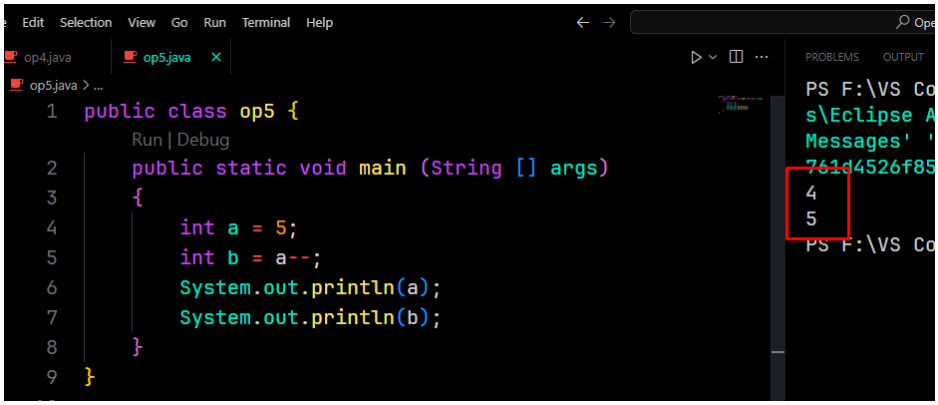
এখানে, a এর value আগে ১ কমবে, তারপর সেটা b তে বসে যাবে।

Post-Increment (++): The value is assigned first and then increments the operand by 1.



এখানে, a এর value আগে b তে বসে যাবে। তারপর a এর value ১ বাড়বে।

Post-Decrement (--): The value is assigned first and then decrements the operand by 1.



এখানে, a এর value আগে b তে বসে যাবে। তারপর a এর value ১ কমবে।

4.8 Operator Precedence

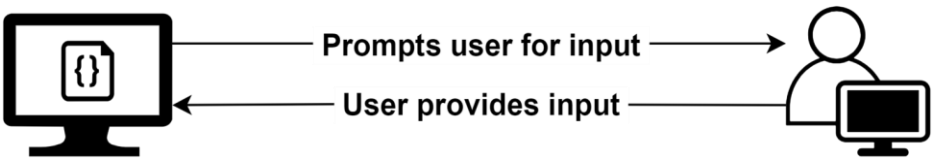
The precedence is higher at the top and decreases as we move towards the bottom. If multiple operators with the same precedence are placed adjacently, operators are executed from left to right. If the first brackets "( )" are present, we must go to the innermost first bracket and gradually move towards the outer ones.

Operator Type	Operator Category/Name	Symbol
Unary	Postfix	++, --
	Prefix	++, --, !, -
Arithmetic	Multiplicative	*, / , %
	Additive	+, -
Relational	Comparison	<, >, <=, >=
	Equality	==, !=
Bitwise	Bitwise AND	&
	Bitwise Exclusive OR	^
	Bitwise Inclusive OR	|
Logical	Logical AND	&&
	Logical OR	||
Assignment	All assignment operators	=, +=, -=, *=, /=, %=

JAVA User Input

User input in Java

- Sometimes we need to take input from the user.
- Creates a connection between the program and it's user.
- Most common way to take input is using the java scanner class.

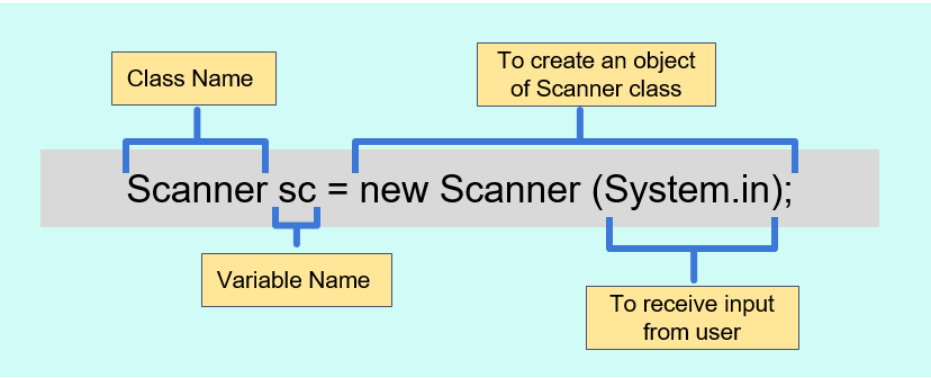


What is a Java Scanner Class?

- A predefined class that reads data from the keyboard or a text file.
- Found in the **java.util** package.
- Can read all types of data (numeric, strings and other data types).
- Easiest way to take input from the user.

User Input using Scanner class

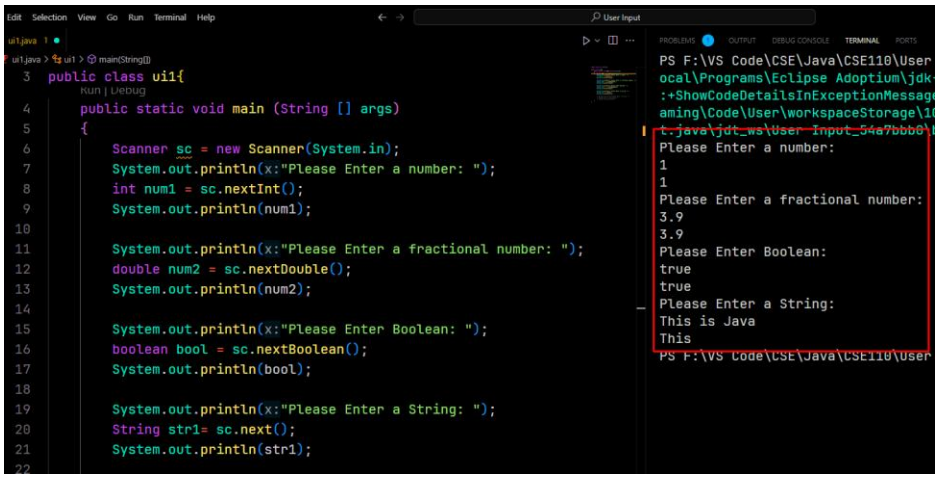
- Import the **java.util**.Scanner package
- Create an object of the Scanner class
- Syntax:



Inputting Different Datatypes

Method	Usage
nextInt( )	Reads an integer value from the user
nextLong( )	Reads a long value from the user
nextFloat( )	Reads a float value from the user

nextDouble( )	Reads a double value from the user
nextBoolean( )	Reads a boolean value from the user
nextLine( )	Reads a string value from the user



```java
public class ui1{
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println(x:"Please Enter a number: ");
        int num1 = sc.nextInt();
        System.out.println(num1);

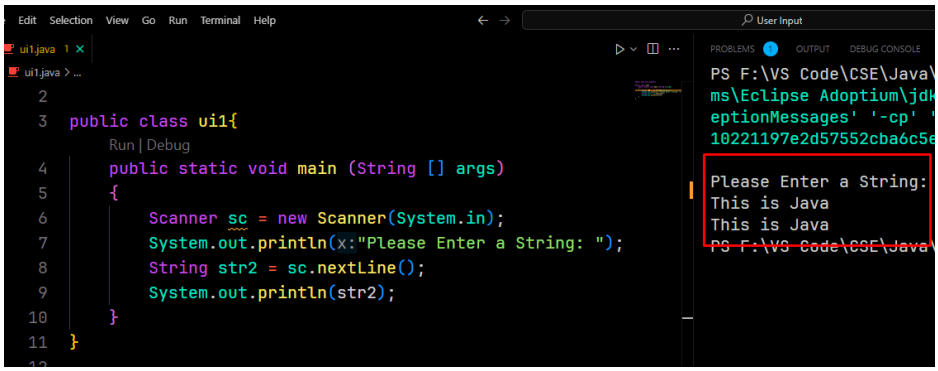
        System.out.println(x:"Please Enter a fractional number: ");
        double num2 = sc.nextDouble();
        System.out.println(num2);

        System.out.println(x:"Please Enter Boolean: ");
        boolean bool = sc.nextBoolean();
        System.out.println(bool);

        System.out.println(x:"Please Enter a String: ");
        String str1= sc.next();
        System.out.println(str1);
```

```
Please Enter a number:
1
1
Please Enter a fractional number:
3.9
3.9
Please Enter Boolean:
true
true
Please Enter a String:
This is Java
This
```

String Input নেয়ার সময় sc.next( ) এর ক্ষেত্রে প্রথম space পর্যন্ত নিবে।



```java
public class ui1{
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println(x:"Please Enter a String: ");
        String str2 = sc.nextLine();
        System.out.println(str2);
    }
}
```

```
Please Enter a String:
This is Java
This is Java
```

sc.nextline( ) এর ক্ষেত্রে সম্পূর্ণ লাইন নিবে।

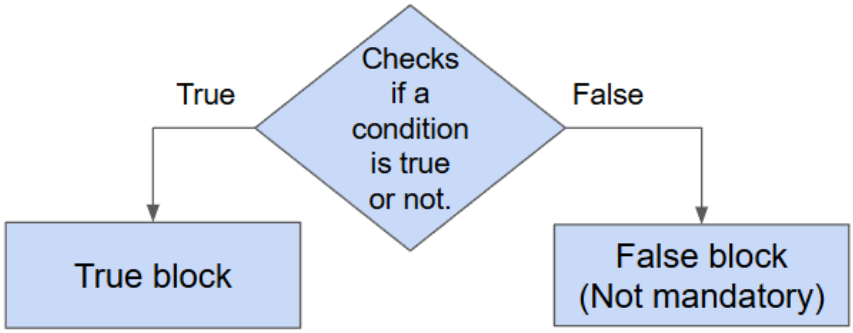
Decision Making

Branching or Selection Statements

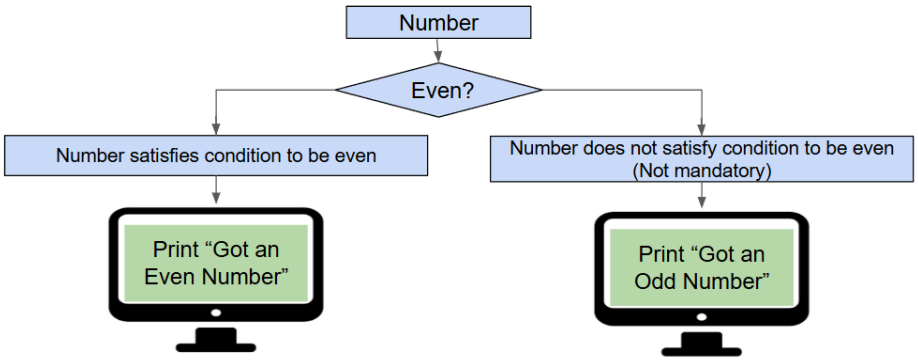
Conditional / Selection Statement

- Execution of a block of code on the basis of some **Conditions**.
- Non sequential program statements
- Selection statements may be **unary or non unary.**

Ex: if, else if, else, switch-case



Ex: Printing “Got an Even Number” only if a number satisfies condition to be even or Print “Got an Odd Number” if a number did not satisfy condition to be even.



অর্থাৎ, ১টা number নিবো এবং যাচাই করবো সেটা Even (জোড়) কিনা। যদি Even (জোড়) হয় তাহলে বলবো Even number পেয়েছি নয়তো বলবো Odd number (বিজোড়) পেয়েছি।

if Selection Statement

- Unary selection statement
- Will execute block of code only if condition is **True**.

```java
import java.util.Scanner;

public class dm1{
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println(x:"Enter a number: ");
        int num = sc.nextInt();
        if (num > 10)
        {
            System.out.println(num);
        }
    }
}
```

```
PS F:\VS Code\CS
ams\Eclipse Adop
tionMessages' '-
Enter a number:
88
88
```

Checking Conditions using Operators

- Used to evaluate expressions and return a boolean value **(true or false).**
- Conditional operators in Java: Equality Operator **(==)**, Inequality Operator **(!=)**, Comparison Operators **(<, >, <=, >=)**, Logical

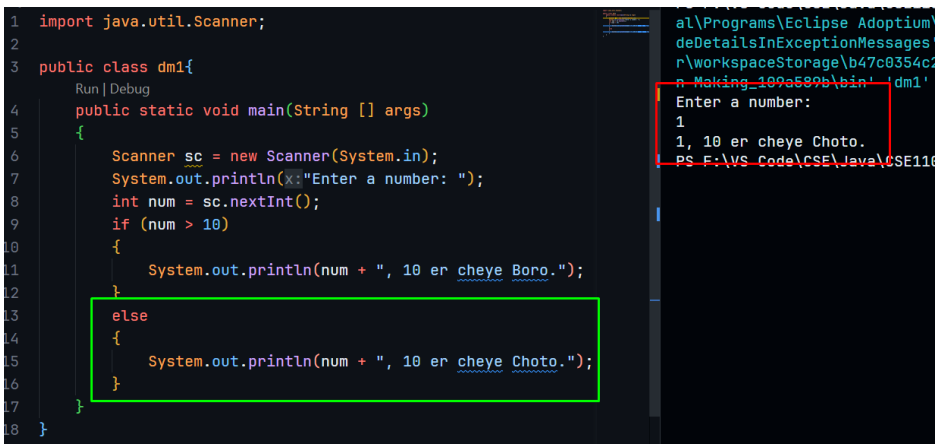
Operators: NOT **(!)**, AND **(&&)**, OR **(||)**.

- Essential for creating effective branching logic.

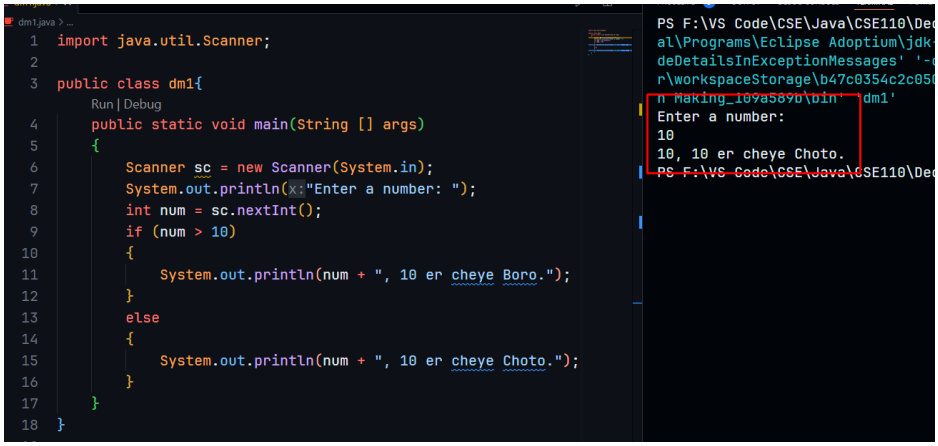
if - else Double Selection Statements

- else block executed when if condition yields **False**.
- else does not have any condition.



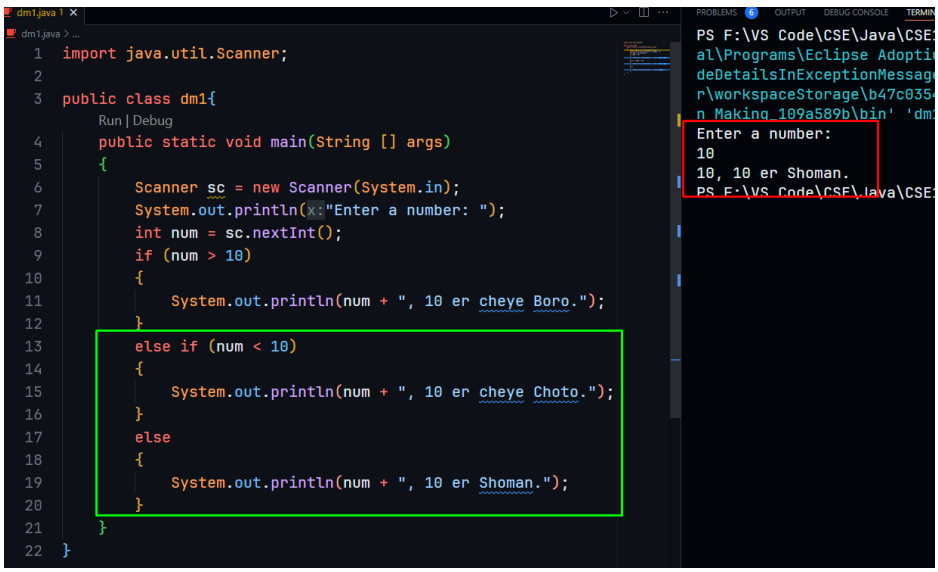


এখানে যদি 10 input দেই তাহলে?



if - else if - else Multiple Selection Statements

- Follows top down approach.
- There can be multiple else if conditions.
- else if and else cannot exist without if condition.
- Only the first True conditional block executed. Following conditions are not checked.
    - Use of elif/ else: Evaluating other conditions after if statement.



Nested Decision Making

- A conditional statement contained inside another conditional statement.
- Statements inside are only evaluated if outer branching statements are true.
    - Use: Building complex layered logic.

Use case: Checking if an even number is a prime number will require nested if

statements. Outer if statement will check if the number is even or not and the inner if will check if the number is prime or not.



```java
import java.util.Scanner;

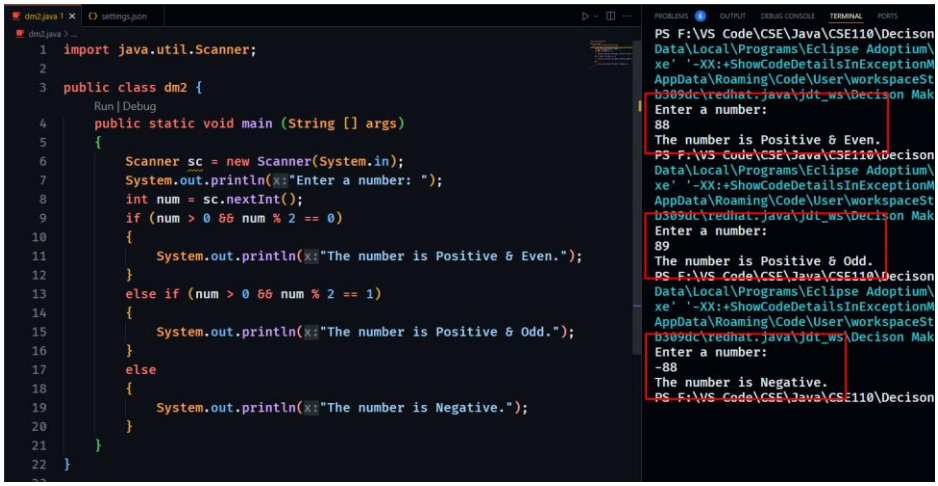
public class dm2 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a number: ");
        int num = sc.nextInt();
        if (num > 0)
        {
            if (num % 2 == 0)
            {
                System.out.println("The number is Positive & Even.");
            }
            else
            {
                System.out.println("The number is Positive & Odd.");
            }
        }
        else
        {
            System.out.println("The number is Negative.");
        }
    }
}
```

```
Enter a number:
88
The number is Positive & Even.
Enter a number:
89
The number is Positive & Odd.
```

এখানে মূলত Condition এর ভিতরে Condition.

Nested না করেই Code টা করা যায় কিনা?

- অবশ্যই যায়।



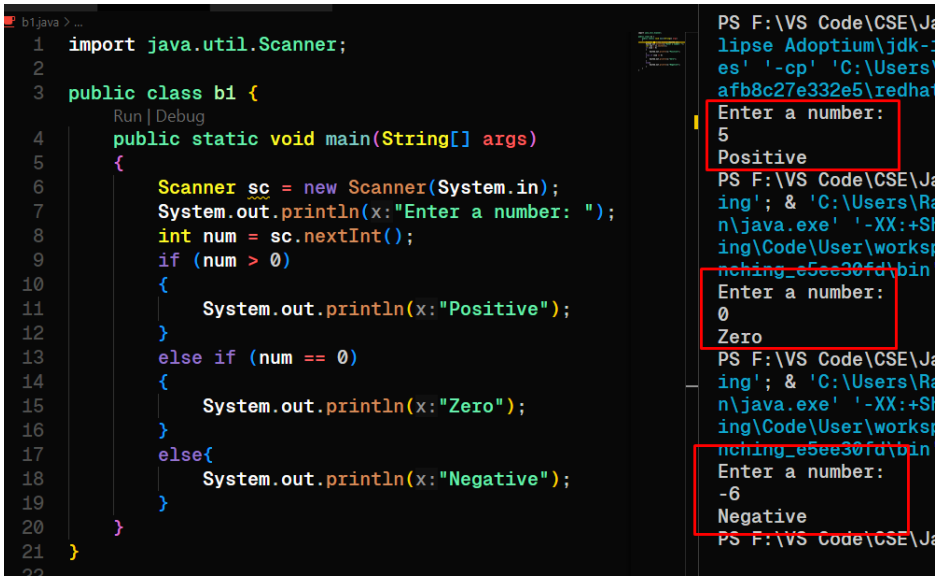
```java
import java.util.Scanner;

public class dm2 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a number: ");
        int num = sc.nextInt();
        if (num > 0 && num % 2 == 0)
        {
            System.out.println("The number is Positive & Even.");
        }
        else if (num > 0 && num % 2 == 1)
        {
            System.out.println("The number is Positive & Odd.");
        }
        else
        {
            System.out.println("The number is Negative.");
        }
    }
}
```

```
Enter a number:
88
The number is Positive & Even.
Enter a number:
89
The number is Positive & Odd.
Enter a number:
-88
The number is Negative.
```

CW

1.

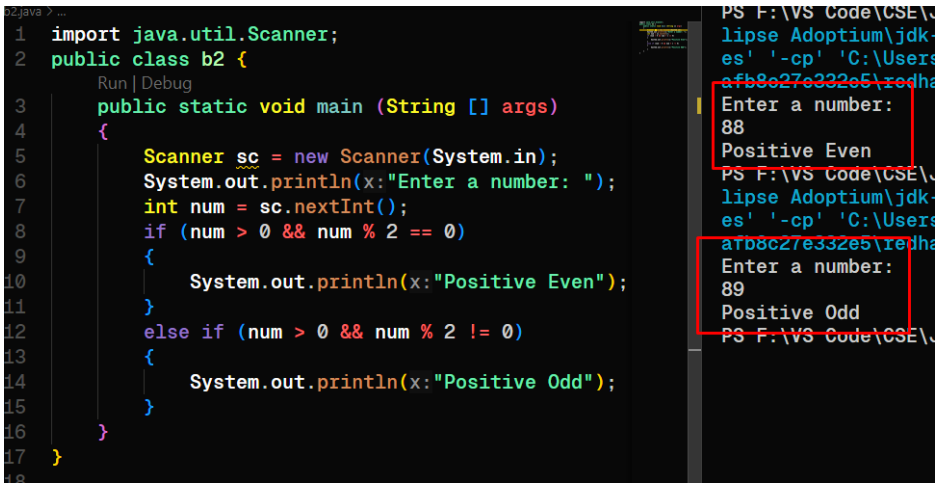


```java
import java.util.Scanner;

public class b1 {
    Run | Debug
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println(x:"Enter a number: ");
        int num = sc.nextInt();
        if (num > 0)
        {
            System.out.println(x:"Positive");
        }
        else if (num == 0)
        {
            System.out.println(x:"Zero");
        }
        else{
            System.out.println(x:"Negative");
        }
    }
}
```

```
PS F:\VS Code\CSE\Ja
lipse Adoptium\jdk-
es' '-cp' 'C:\Users
afb8c27e332e5\redha
Enter a number:
5
Positive
PS F:\VS Code\CSE\Ja
ing'; & 'C:\Users\Ra
n\java.exe' '-XX:+Sh
ing\Code\User\works
nching_e5ee30fd\bin
Enter a number:
0
Zero
PS F:\VS Code\CSE\Ja
ing'; & 'C:\Users\Ra
n\java.exe' '-XX:+Sh
ing\Code\User\works
nching_e5ee30fd\bin
Enter a number:
-6
Negative
PS F:\VS Code\CSE\Ja
```

2.



```java
import java.util.Scanner;
public class b2 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println(x:"Enter a number: ");
        int num = sc.nextInt();
        if (num > 0 && num % 2 == 0)
        {
            System.out.println(x:"Positive Even");
        }
        else if (num > 0 && num % 2 != 0)
        {
            System.out.println(x:"Positive Odd");
        }
    }
}
```

```
lipse Adoptium\jdk-
es' '-cp' 'C:\Users
Enter a number:
88
Positive Even
lipse Adoptium\jdk-
es' '-cp' 'C:\Users
Enter a number:
89
Positive Odd
```

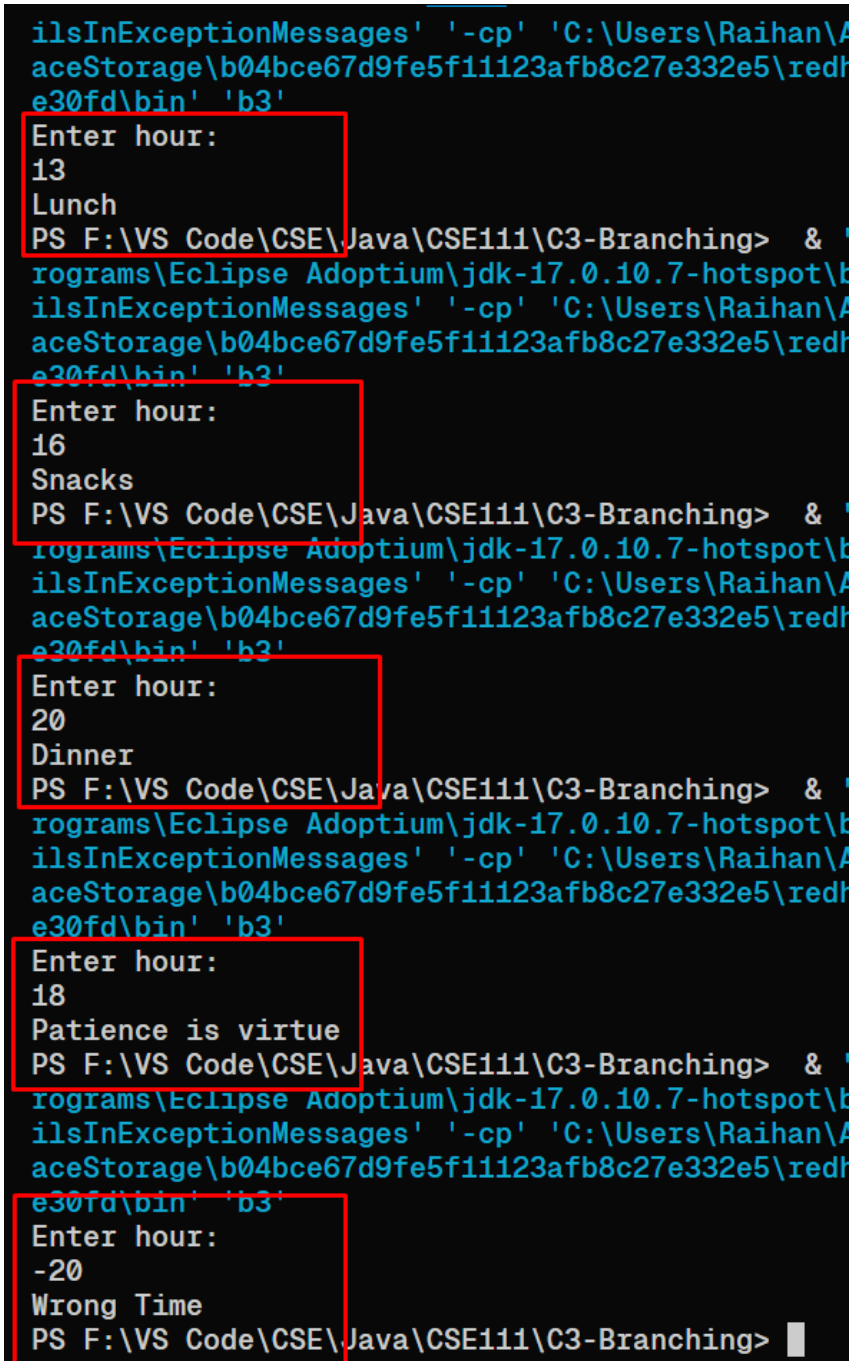
3.



```java
import java.util.Scanner;

public class b3 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter hour: ");
        int hr = sc.nextInt();
        if (hr >= 0 && hr <= 23)
        {
            if (hr >= 4 && hr <= 6)
            {
                System.out.println("Breakfast");
            }
            else if (hr >= 12 && hr <= 13)
            {
                System.out.println("Lunch");
            }
            else if (hr >= 16 && hr <= 17)
            {
                System.out.println("Snacks");
            }
            else if (hr >= 19 && hr <= 20)
            {
                System.out.println("Dinner");
            }
            else
            {
                System.out.println("Patience is virtue");
            }
        }
        else
        {
            System.out.println("Wrong Time");
        }

    }
}
```


ilsInExceptionMessages' '-cp' 'C:\Users\Raihan\A
aceStorage\b04bce67d9fe5f11123afb8c27e332e5\redh
e30fd\bin' 'b3'
Enter hour:
13
Lunch
PS F:\VS Code\CSE\Java\CSE111\C3-Branching> & '
rograms\Eclipse Adoptium\jdk-17.0.10.7-hotspot\b
ilsInExceptionMessages' '-cp' 'C:\Users\Raihan\A
aceStorage\b04bce67d9fe5f11123afb8c27e332e5\redh
e30fd\bin' 'b3'
Enter hour:
16
Snacks
PS F:\VS Code\CSE\Java\CSE111\C3-Branching> & '
rograms\Eclipse Adoptium\jdk-17.0.10.7-hotspot\b
ilsInExceptionMessages' '-cp' 'C:\Users\Raihan\A
aceStorage\b04bce67d9fe5f11123afb8c27e332e5\redh
e30fd\bin' 'b3'
Enter hour:
20
Dinner
PS F:\VS Code\CSE\Java\CSE111\C3-Branching> & '
rograms\Eclipse Adoptium\jdk-17.0.10.7-hotspot\b
ilsInExceptionMessages' '-cp' 'C:\Users\Raihan\A
aceStorage\b04bce67d9fe5f11123afb8c27e332e5\redh
e30fd\bin' 'b3'
Enter hour:
18
Patience is virtue
PS F:\VS Code\CSE\Java\CSE111\C3-Branching> & '
rograms\Eclipse Adoptium\jdk-17.0.10.7-hotspot\b
ilsInExceptionMessages' '-cp' 'C:\Users\Raihan\A
aceStorage\b04bce67d9fe5f11123afb8c27e332e5\redh
e30fd\bin' 'b3'
Enter hour:
-20
Wrong Time
PS F:\VS Code\CSE\Java\CSE111\C3-Branching>

Loops (while)

Intro to loop

Loop is one of fundamental concepts of programming.

It is a **control structure** that allows one to repeatedly execute a block of codes.

Components of a loop:

1) **Initialization:** one or more variable(s) to keep track of the loop.
2) **Condition:** boolean expression based on which the loop runs.
3) **Body:** block of codes which is executed each time the loop runs.
4) **Steps:** updating the initialized variable(s) to make sure the loop runs certain amount of times only.

1, 2, 4 পরস্পরের উপর নির্ভরশীল।

Loops in Java

There are three types of loops in Java.

1. while loop.
2. for loop.
3. do while loop.

◆ **for and while** loop only enters the loop's body if the condition is true.
◆ On the other hand, **do while** loop executes the body's code first then checks the condition.

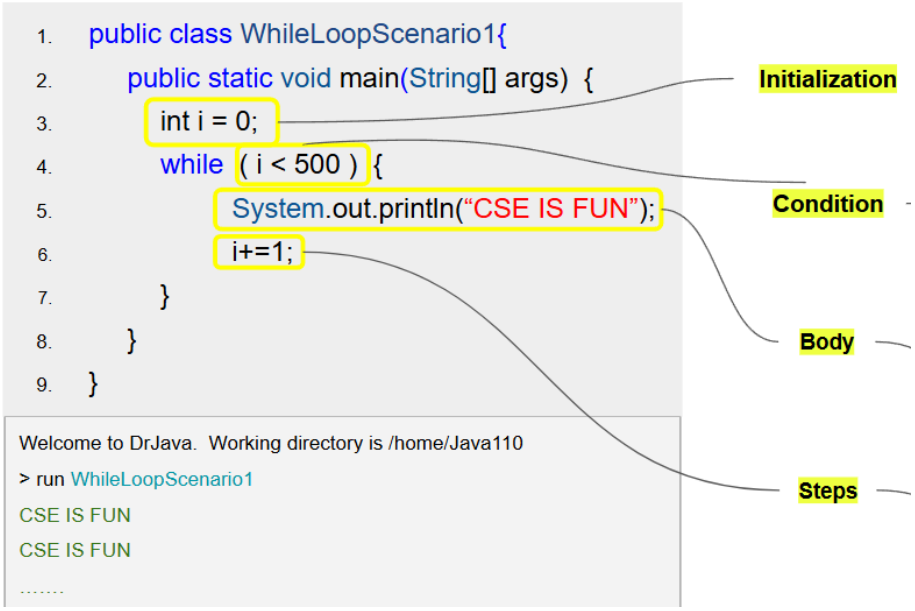
while loop

Structure of **while loop**:

```
1.  INITIALIZATION
2.  while ( CONDITION ){
3.      BODY OF THE LOOP
4.      STEPS
5.  }
```

Key points of the structure:

- We **initialize** the variable outside the loop's body.
- After the **while** keyword we write the **condition** within parenthesis.
- Then at the end of the loop's body we **increase/decrease** the initialized variable.

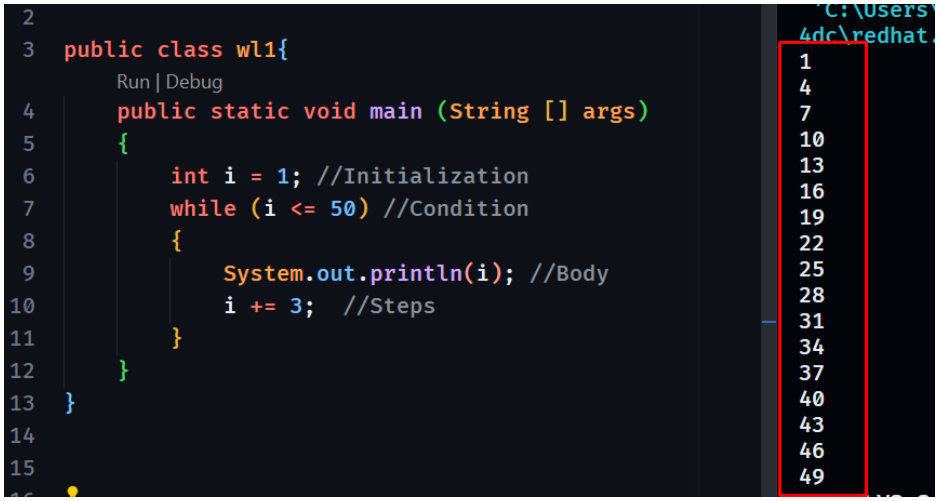



```java
public class wl1{
    Run | Debug
    public static void main (String [] args)
    {
        int i = 0; //Initialization
        while (i < 10) //Condition
        {
            System.out.println(x:"CSE 110"); //Body
            i += 1;  //Steps
        }
    }
}
```

```
f46059071e6cba16
CSE 110
CSE 110
CSE 110
CSE 110
CSE 110
CSE 110
CSE 110
CSE 110
CSE 110
CSE 110
PS F:\VS Code\CS
```

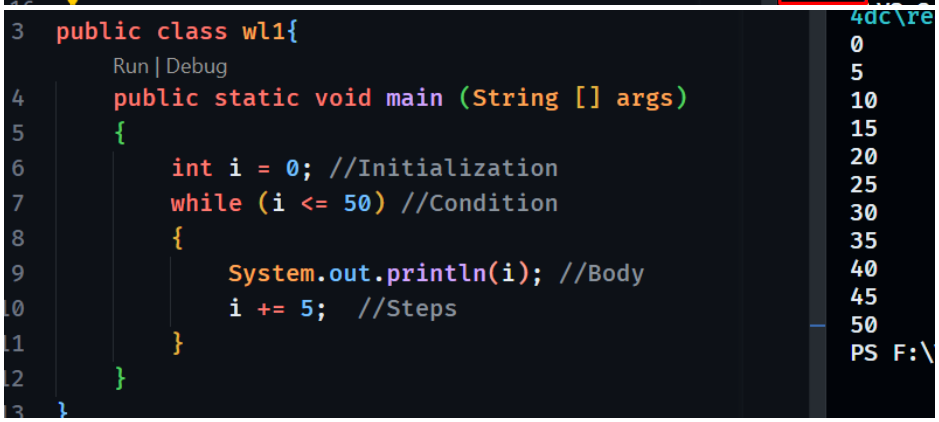
```java
public class wl1{
    Run | Debug
    public static void main (String [] args)
    {
        int i = 5; //Initialization
        while (i >= 1) //Condition
        {
            System.out.println(i); //Body
            i -= 1;  //Steps
        }
    }
}
```

```
5
4
3
2
1
PS F:\VS Code
```



```java
public class wl1{
    Run | Debug
    public static void main (String [] args)
    {
        int i = 1; //Initialization
        while (i <= 50) //Condition
        {
            System.out.println(i); //Body
            i += 3;  //Steps
        }
    }
}
```

```
C:\Users
4dc\redhat.
1
4
7
10
13
16
19
22
25
28
31
34
37
40
43
46
49
```



```java
public class wl1{
    Run | Debug
    public static void main (String [] args)
    {
        int i = 0; //Initialization
        while (i <= 50) //Condition
        {
            System.out.println(i); //Body
            i += 5;  //Steps
        }
    }
}
```

```
4dc\re
0
5
10
15
20
25
30
35
40
45
50
PS F:\
```

CW

1.

```java
public class l1{
    Run | Debug
    public static void main (String [] args)
    {
        int num = 2;
        while (num <= 10)
        {
            System.out.print(num + " ");
            num += 2;
        }

    }
}
```

```
Adoptium\jdk-1
' 'C:\Users\Ra
0d1\redhat.jav
2 4 6 8 10
PS F:\VS Code\
```

2.

```java
public class l2 {
    Run | Debug
    public static void main (String [] args)
    {
        int num = 5;
        while (num <= 30)
        {
            if (num % 2 == 0)
            {
                System.out.print("-" + num + " ");
            }
            else
            {
                System.out.print(num + " ");
            }
        num += 5;
        }
    }
}
```

```
PS F:\VS Code\CSE\Jav
clipse Adoptium\jdk-1
Messages' '-cp' 'C:\U
d0ca76d1ba65d0a7ecb9c
5 -10 15 -20 25 -30
PS F:\VS Code\CSE\Jav
```

Loops (for loop)

for loop

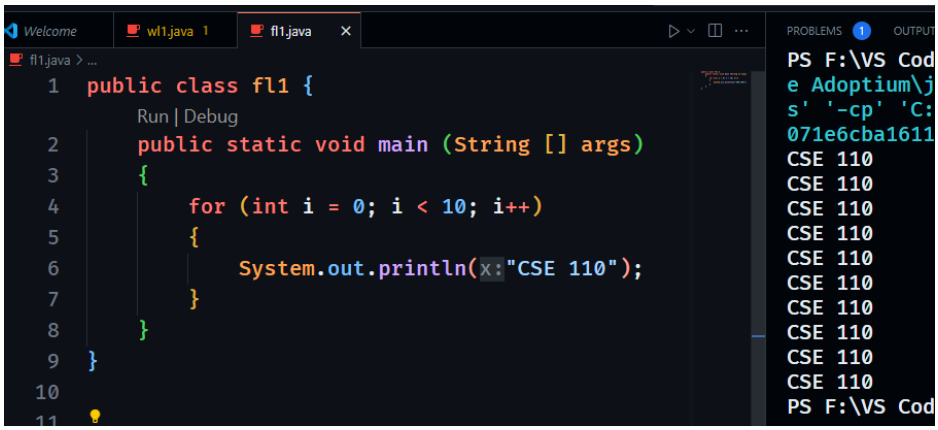
Structure of **for loop**:

```
1.  for ( INITIALIZATION ; CONDITION ; STEPS ) {
2.          BODY OF THE LOOP
3.  }
```

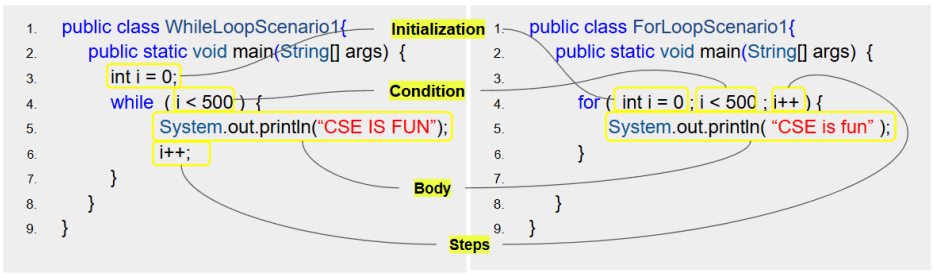
Key points of the structure:

- We **initialize** variable, write the **condition** and **increase/decrease** the initialized variable within a parenthesis right after the **for** keyword.
- The aforementioned 3 components are separated using **;** (**semicolon**) .
- We can initialize or increase/decrease multiple variables at once by separating them using **,** (**Comma**).

Scenario: Write a java program using for loop to print “CSE 110” 10 times.

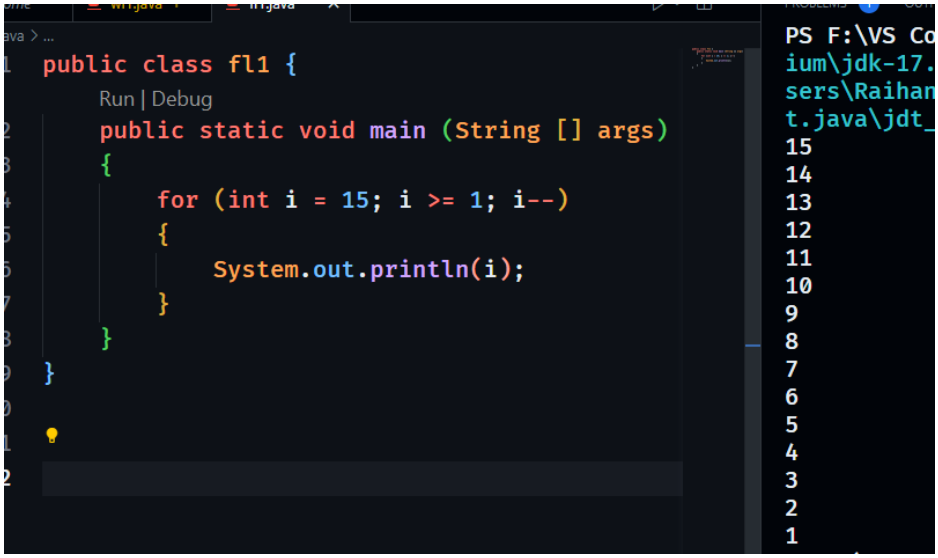


for loop vs while loop





◆ 15 থেকে 1 পর্যন্ত উল্টা Print করতে চাইলে →



```java
public class fl1 {
    Run | Debug
    public static void main (String [] args)
    {
        for (int i = 15; i >= 1; i--)
        {
            System.out.println(i);
        }
    }
}
```

```
PS F:\VS Co
ium\jdk-17.
sers\Raihan
t.java\jdt_
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
```

◆ ১ থেকে ৫০ পর্যন্ত Even number (জোড় সংখ্যা) Print করতে চাইলে-



◆ User ৫ টা number দিবে, সেখান থেকে maximum value অর্থাৎ সবচেয়ে বড় সংখ্যা বের করতে হবে।



```java
import java.util.Scanner;

public class fl2 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        int maxVal = 0;

        for (int i = 1; i <= 5; i++)
        {
            System.out.println(x:"Enter a number: ");
            int num = sc.nextInt();
            if (i == 1)
            {
                maxVal = num;
            }
            else
            {
                if (num > maxVal)
                {
                    maxVal = num;
                }
            }
        }
        System.out.println(maxVal + " is the maximum.");
    }
}
```

```
PS F:\VS Code\CSE\Java\CS
Eclipse Adoptium\jdk-17.0
ptionMessages' '-cp' 'C:\
\18a4976ec9df46059071e6cb
Enter a number:
9
Enter a number:
5
Enter a number:
17
Enter a number:
19
Enter a number:
88
88 is the maximum.
PS F:\VS Code\CSE\Java\CS
```

CW

1.

```java
public class l3 {
    Run | Debug
    public static void main (String [] args)
    {
        for (int i = 2; i <= 10; i+=2)
        {
            System.out.print(i+ " ");
        }
    }
}
```

```
PS F:\VS Cod
clipse Adopt
Messages' '-
d6ca76d1ba65
2 4 6 8 10
PS F:\VS Cod
```

2.

```java
public class l4 {
    Run | Debug
    public static void main (String [] args)
    {
        for (int i = 5; i <= 30; i += 5)
        {
            if(i % 2 == 0)
            {
            System.out.print(-i+ " ");
            }
            else
            {
                System.out.print(i + " ");
            }
        }
    }
}
```

```
optium\jdk-17.0.10.7-
:\Users\Raihan\AppDa
dbat_java\jdt_ws\C4-
5 -10 15 -20 25 -30
PS F:\VS Code\CSE\Ja
```

Loops (break/continue)

Two important concepts one should learn with loop are **break** & **continue**. These two keywords grant us more control when running the loop.

- **break:** the break keyword allows us to **stop the loop** early.
- **continue:** the continue keyword allows us to skip the rest of the loop body and start the next iteration of the loop.

Note: both break and continue are written inside the loop's body and are generally bounded by a condition.

break keyword – code

```
1.  public class BreakScenario4{
2.      public static void main(String[] args) {
3.          for ( int i=1 ; i<=10 ; i++ ){
4.              if ( i == 5 ) {
5.                  break;
6.              }
7.              System.out.print( i + " " );
8.          }
9.      }
10. }
```

Initialization
Condition
Steps
Break
Body

```
Welcome to DrJava.  Working directory is /home/Java110
> run BreakScenario4
1 2 3 4 >
```

◆ break এর মূল কাজ হলো একটা Loop কে ভেঙ্গে দেওয়া।

```
1  import java.util.Scanner;
2
3  public class bc {
       Run | Debug
4      public static void main (String [] args)
5      {
6          Scanner sc = new Scanner(System.in);
7          System.out.println("Jeta Dorkar: ");
8          int id = sc.nextInt();
9          for (int i = 1; i <= 10; i++)
10         {
11             if (i == id)
12             {
13                 System.out.println(id + " khuje peyechi");
14                 break;
15             }
16             System.out.println(i);
17         }
18     }
19 }
```

```
Jeta Dorkar:
5
1
2
3
4
5 khuje peyechi
```

এখানে 5 দরকার ছিল, তাই 5 পেয়ে যাবার পর break হয়ে Loop এর বাইরে চলে এসেছে।

continue keyword – code

```
1.  public class ContinueScenario4{
2.      public static void main(String[] args) {
3.          for ( int i=1 ; i<=10 ; i++ ){
4.              if ( i == 5 ) {
5.                  continue;
6.              }
7.              System.out.print( i + " " );
8.          }
9.      }
10. }
```

Initialization

Condition

Steps

Continue

Body

```
Welcome to DrJava.  Working directory is /home/Java110
> run ContinueScenario4
1 2 3 4 6 7 8 9 10 >
```

```
bc2.java > ...
1  import java.util.Scanner;
2
3  public class bc2 {
       Run | Debug
4      public static void main (String [] args)
5      {
6          Scanner sc = new Scanner(System.in);
7          for (int i = 1; i <= 10; i++)
8          {
9              if (i == 6 || i == 8)
10             {
11                 continue;
12             }
13             System.out.println(i + " --> chocolate given.");
14         }
15     }
16 }
```

```
PS F:\VS Code\CSE\Java\C
ams\Eclipse Adoptium\jdk
lsInExceptionMessages' '
spaceStorage\18a4976ec9d
0d4\bin' 'bc2'
1 --> chocolate given.
2 --> chocolate given.
3 --> chocolate given.
4 --> chocolate given.
5 --> chocolate given.
7 --> chocolate given.
9 --> chocolate given.
10 --> chocolate given.
PS F:\VS Code\CSE\Java\C
```

এখানে 6 & 8 কে chocolate দিবো না তাই continue অর্থাৎ 6 & 8 কে বাদ দিয়ে Loop এ ঘুরবে।

Nested Loops

Key points of the structure:

- We write a brand new loop inside the body of another loop.
- In the nested loop we need to make sure to use different variable than the one we used in the outer loop.
- For each iteration of the outer loop, the inner loop executes all its iteration.

Structure of **nested for loop**:

```
1.  for ( INITIALIZATION_1 ; CONDITION_1 ; STEPS_1 ) {
2.          for ( INITIALIZATION_2 ; CONDITION_2 ; STEPS_2 ) {
3.                  BODY OF THE LOOP
4.          }
5.  }
```

while loop দিয়ে –

```java
public class nes1 {
    Run | Debug
    public static void main(String [] args)
    {
        int outer = 1;
        while(outer <= 2) // 1 <= 2
        {
            int inner = 1;
            while (inner <= 3) // 1 <= 3
            {
                System.out.println(outer + " " + inner); // 1 1
                inner += 1; // 1 + 1 = 2
            }
            System.out.println(x:"Inner Loop Shesh");
            outer += 1; // 1 + 1 = 2
        }
    }
}
```

```
de\User\workspaceStor
t_ws\Loops_912140d4\b
1 1
1 2
1 3
Inner Loop Shesh
2 1
2 2
2 3
Inner Loop Shesh
PS F:\VS Code\CSE\Jav
```

for loop দিয়ে-

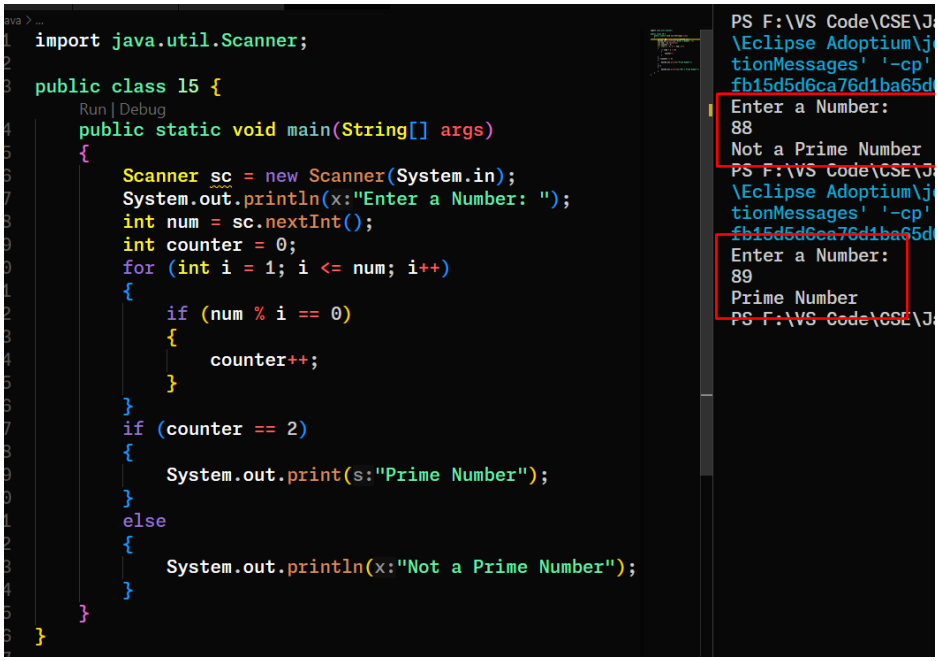
```java
public class nes2 {
    Run | Debug
    public static void main(String [] args)
    {
        for (int outer = 1; outer <= 2; outer++) // 1<=2; 1+1=2 --> 2<=2
        {
            for (int inner = 1; inner <= 3; inner++) // 1<=3; 1+1=2 --> 2<=3
            {
                System.out.println(outer + " " + inner); // 1 1     1 2
            }
            System.out.println(x:"Inner Loop Shesh");
        }
    }
}
```

```
spaceStorage\18a49
\jdt_ws\Loops_9121
1 1
1 2
1 3
Inner Loop Shesh
2 1
2 2
2 3
Inner Loop Shesh
PS F:\VS Code\CSE\
```

Practice Problem



3. Take an integer number from the user and check if it is a Prime or not. If it is a Prime then print “Prime Number” otherwise “Not a Prime Number.”



```java
import java.util.Scanner;

public class l5 {
    Run | Debug
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println(x:"Enter a Number: ");
        int num = sc.nextInt();
        int counter = 0;
        for (int i = 1; i <= num; i++)
        {
            if (num % i == 0)
            {
                counter++;
            }
        }
        if (counter == 2)
        {
            System.out.print(s:"Prime Number");
        }
        else
        {
            System.out.println(x:"Not a Prime Number");
        }
    }
}
```

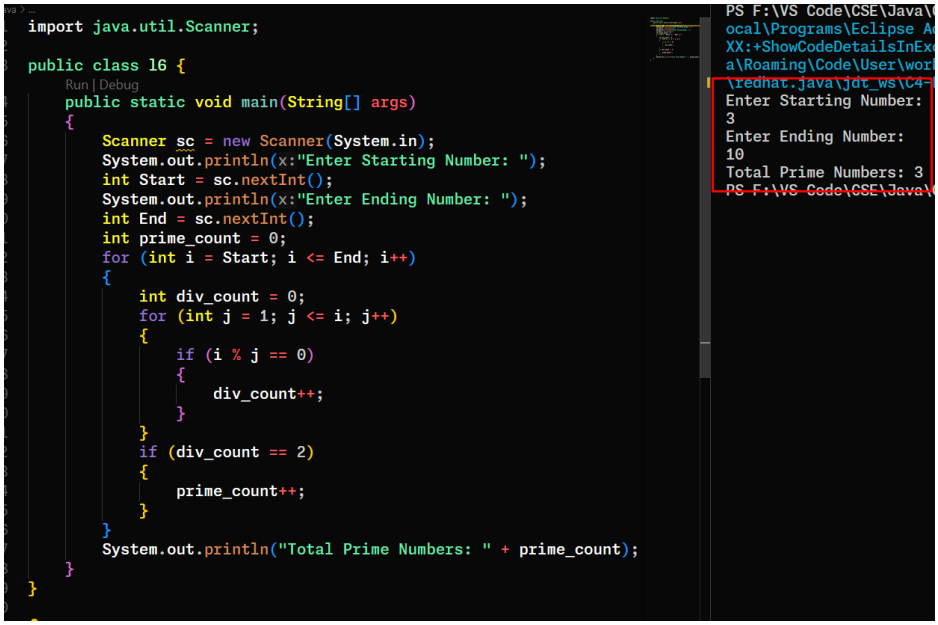
```
PS F:\VS Code\CSE\J
\Eclipse Adoptium\j
tionMessages' '-cp'
fb15d5d6ca76d1ba65d
Enter a Number:
88
Not a Prime Number
PS F:\VS Code\CSE\J
\Eclipse Adoptium\j
tionMessages' '-cp'
fb15d5d6ca76d1ba65d
Enter a Number:
89
Prime Number
PS F:\VS Code\CSE\J
```

Prime Number = মৌলিক সংখ্যা।

যে সংখ্যাকে ১ এবং ঐ সংখ্যা ছাড়া আর অন্য কোনো সংখ্যা দিয়ে ভাগ করা যায় না সেগুলোই মৌলিক সংখ্যা।

যেমনঃ ৫।

4. Take 2 integer numbers from the user and print how many Prime numbers are there between those numbers.

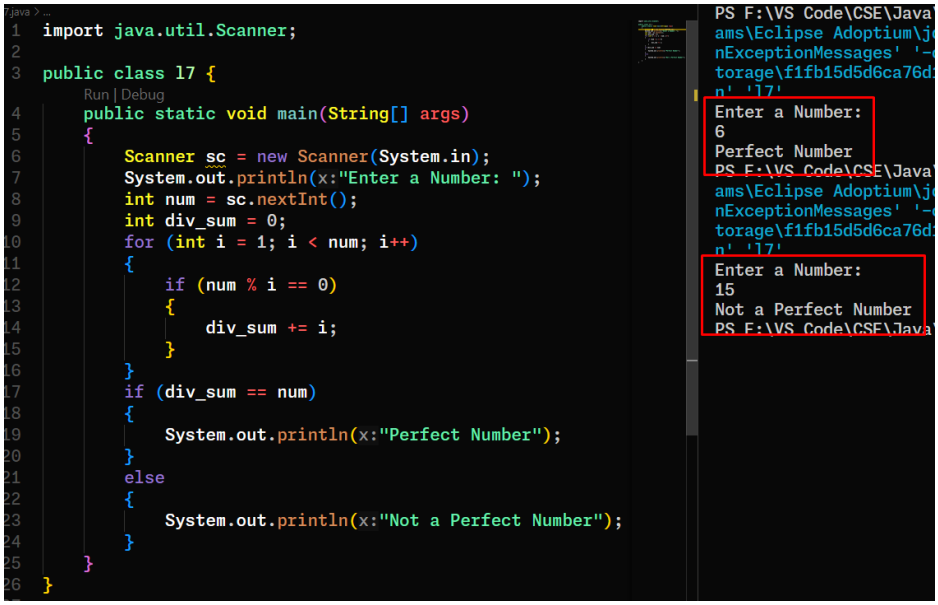


```java
import java.util.Scanner;

public class l6 {
    Run | Debug
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println(x:"Enter Starting Number: ");
        int Start = sc.nextInt();
        System.out.println(x:"Enter Ending Number: ");
        int End = sc.nextInt();
        int prime_count = 0;
        for (int i = Start; i <= End; i++)
        {
            int div_count = 0;
            for (int j = 1; j <= i; j++)
            {
                if (i % j == 0)
                {
                    div_count++;
                }
            }
            if (div_count == 2)
            {
                prime_count++;
            }
        }
        System.out.println("Total Prime Numbers: " + prime_count);
    }
}
```

```
PS F:\VS Code\CSE\Java\
ocal\Programs\Eclipse A
XX:+ShowCodeDetailsInEx
a\Roaming\Code\User\wor
\redhat.java\jdt_ws\C4-
Enter Starting Number:
3
Enter Ending Number:
10
Total Prime Numbers: 3
PS F:\VS Code\CSE\Java\
```

5. Take a number from the user and check if it is Perfect or Not.



```java
import java.util.Scanner;

public class l7 {
    Run | Debug
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println(x:"Enter a Number: ");
        int num = sc.nextInt();
        int div_sum = 0;
        for (int i = 1; i < num; i++)
        {
            if (num % i == 0)
            {
                div_sum += i;
            }
        }
        if (div_sum == num)
        {
            System.out.println(x:"Perfect Number");
        }
        else
        {
            System.out.println(x:"Not a Perfect Number");
        }
    }
}
```

```
PS F:\VS Code\CSE\Java
ams\Eclipse Adoptium\j
nExceptionMessages' '-
torage\f1fb15d5d6ca76d
n' 'l7'
Enter a Number:
6
Perfect Number
PS F:\VS Code\CSE\Java
ams\Eclipse Adoptium\j
nExceptionMessages' '-
torage\f1fb15d5d6ca76d
n' 'l7'
Enter a Number:
15
Not a Perfect Number
PS F:\VS Code\CSE\Java
```

Perfect Number = কোনো সংখ্যার গুণনীয়ক কিংবা ভাজক (Divisor) গুলোর সমষ্টি যদি ওই সংখ্যার সমান হয় [ওই সংখ্যা বাদে] তখন সেটা Perfect Number.

যেমনঃ ৬ এর গুণনীয়ক কিংবা ভাজক (Divisor) গুলো হলো ১, ২, ৩, ৬। ৬ বাদে বাকিগুলো অর্থাৎ, ১+২+৩=৬। তাই ৬ একটি Perfect Number.

Strings

What is a string?

- In many other programming languages, string is a sequence of characters.
- In Java, strings are objects.
- Strings are surrounded by double quotes.
- Strings are immutable. Once a string is created, it can't be changed.
- If we want to modify a string, it will create a new string which contains the modifications.

String Input and Output

- .next( ) and .nextLine( ) of the scanner class is used to take string as input.
- .next( ) only takes input until it finds a " " or space in the given input.
- .nextLine( ) takes input until it finds a "\n" in the given input.



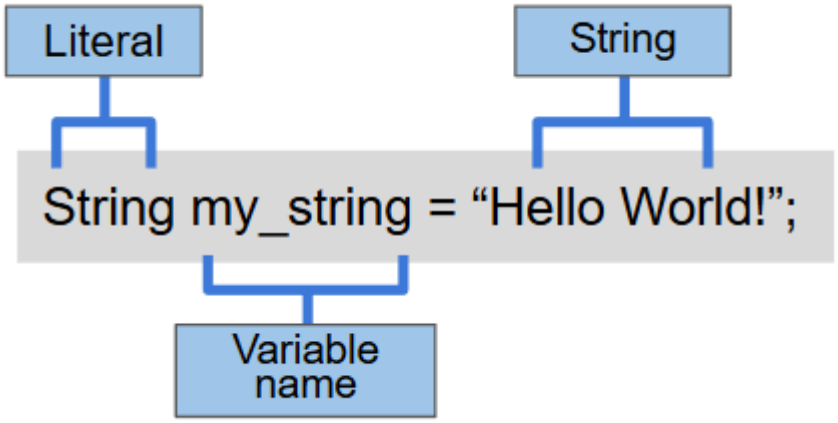
String Input নেয়ার সময় sc.next( ) এর ক্ষেত্রে প্রথম space পর্যন্ত নিবে।



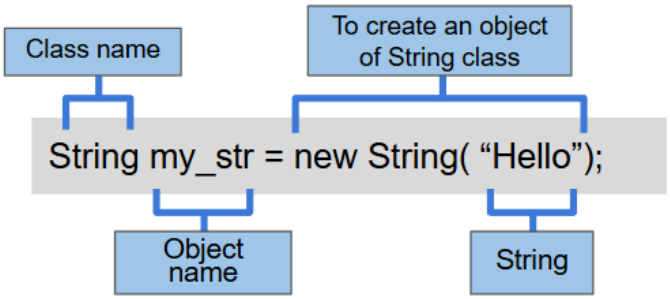
sc.nextline( ) এর ক্ষেত্রে সম্পূর্ণ লাইন নিবে।

String Initialization

- Initialization using String literals:

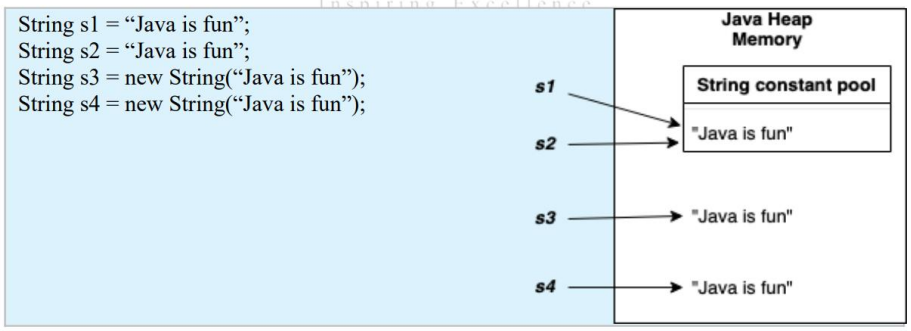


- Creating an object of String class:



Using the new keyword creates a new String object each time in the heap memory.

Let’s write these 4 lines of code and see how the string objects are stored in the diagram below.



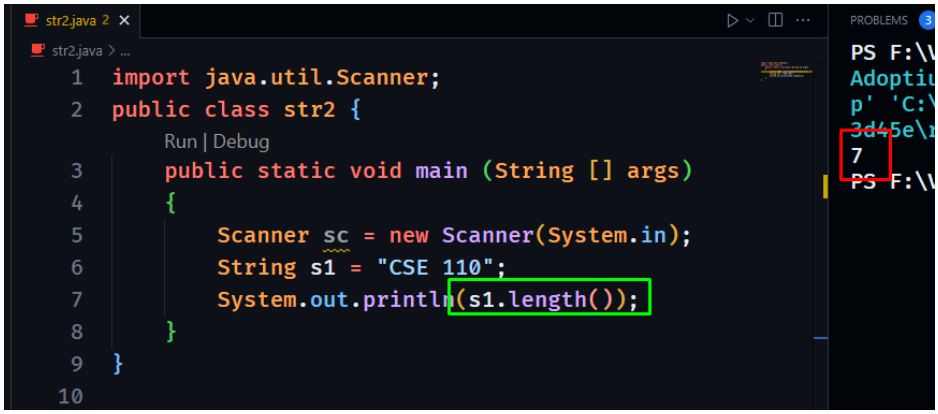
When s1 is created, there is no String literal “Java is fun” in the string constant pool and so it is added there.
When s2 is created, the string “Java is fun” is already present in the pool and hence its reference is only returned to s2, and that is why s1 and s2 both point at the same String object. But s3 and s4 are created using the String constructor and so both

created new String objects which reside in the heap and both have separate locations.

String Length

- The number of characters in a string.
- Can find using the length( ) method.
- String indexing starts from 0 and ends at (Length of string - 1). [০ থেকে শুরু হলে মোট যতগুলো আছে তার থেকে ১টি কম হয়। যেমনঃ ১ টা String এ যদি ৭ টা character থাকে তাহলে তার মোট index হবে ০ থেকে ৬]
- If we create string1 = “CSE IS FUN.”, which has a length of 11, the indexing will look like this:

0	1	2	3	4	5	6	7	8	9	10
C	S	E		I	S		F	U	N	.

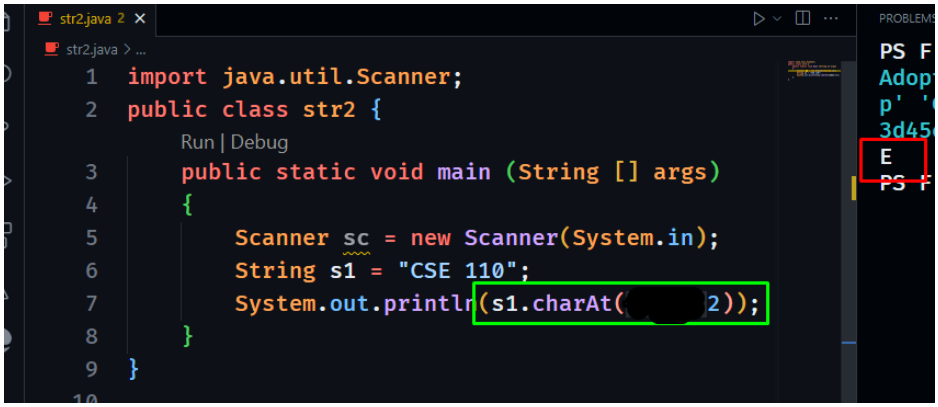


- Blanks and special characters are also counted as characters. (ফাঁকা জায়গা/Space ও ১ টা Character হিসেবে গণ্য হবে।)

.charAt(index) Method

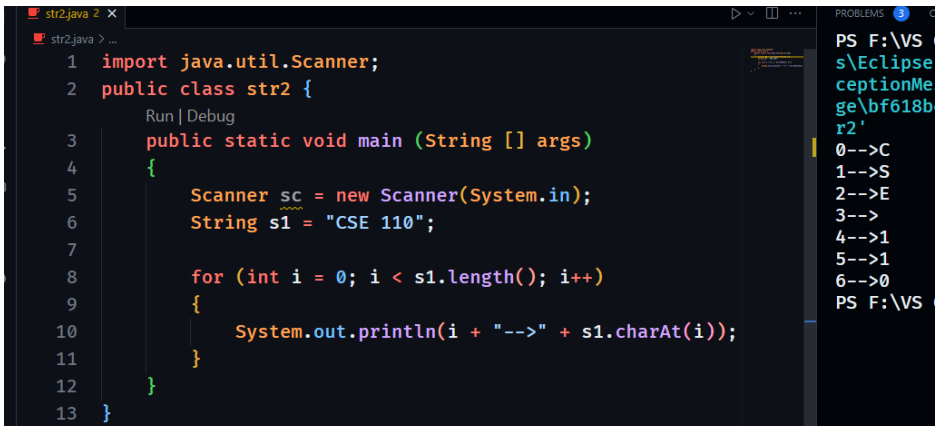
- .charAt(index)
  - Returns the character at index value passed as argument.

- We can get characters at each index by using **charAt(index)** method. Example: string1.charAt(2) will return “E”.



Iterating a String

- Accessing the characters of a string one by one.
- Iterating a string using for loop and charAt( ) method.



এখানে যেই String নিয়েছি loop এর মাধ্যমে সেটার index অনুযায়ী character গুলো বের করে এনেছি।

আর এখানে **i<s1.length()/i<=s1.length()-1,** কেননা ০ থেকে শুরু করে index, total length এর আগে পর্যন্ত যাবে।

ASCII values

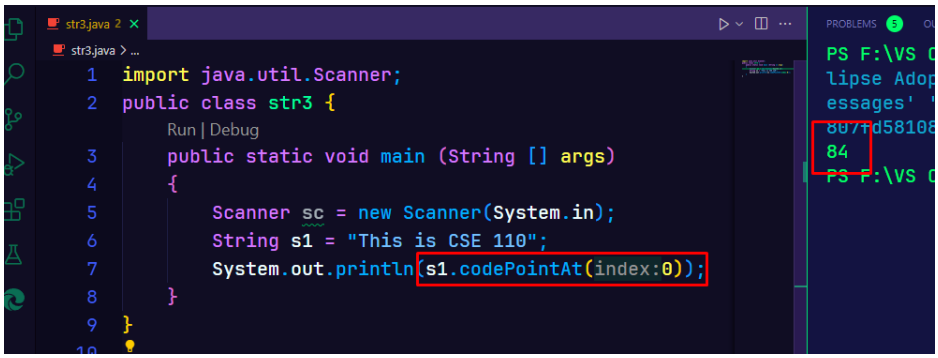
- Unique Unicode value assigned to all keyboard characters.
- Range of ASCII values of **uppercase characters ('A' to 'Z'):** **65 - 90**
- Range of ASCII values of **lowercase characters ('a' to 'z'):** **97 - 122**
- ASCII value of space: **32**
- ASCII value of (0-9): **48 – 57**

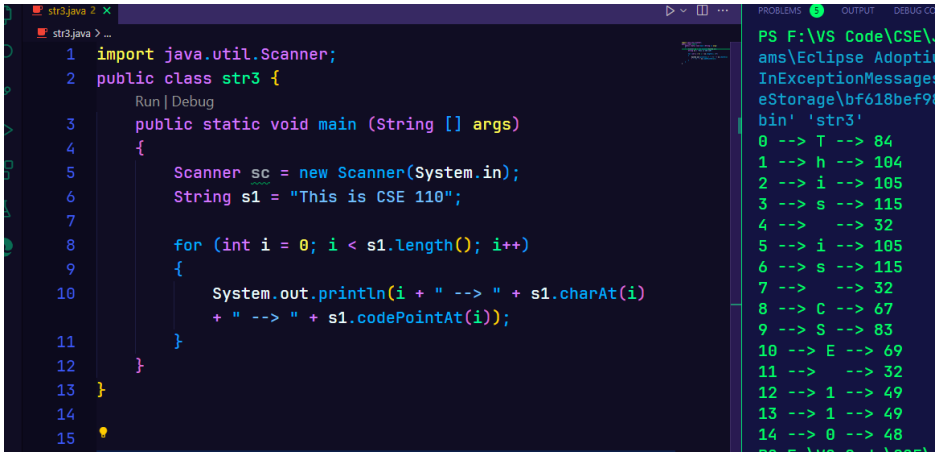
```
12
13 // A to Z - 65 to 90
14 // A - 65
15 // B - 66
16
17 // a to z - 97 to 122
18 // a - 97
19 // b - 98
20
21 // 'a' 97-32 = 65 'A'
22 // 'b' 98-32 = 66 'B'
23
24 // 'A' 65+32 = 97 'a'
25 // 'B' 66+32 = 98 'b'
26
27
```

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	NUL (null)	32	SPACE	64	@	96	`
1	SOH (start of heading)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	TAB (horizontal tab)	41	)	73	I	105	i
10	LF (NL line feed, new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (NP form feed, new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (device control 1)	49	1	81	Q	113	q
18	DC2 (device control 2)	50	2	82	R	114	r
19	DC3 (device control 3)	51	3	83	S	115	s
20	DC4 (device control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of trans. block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	|
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL

Converting a Character to ASCII Value

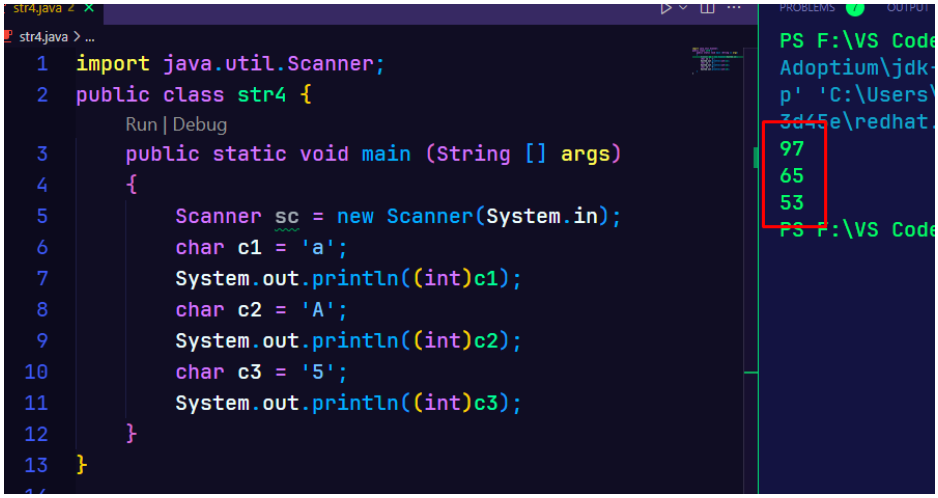
- .codePointAt(index)
  - Takes an index of a string as a parameter and returns the ASCII value of the character of that index.





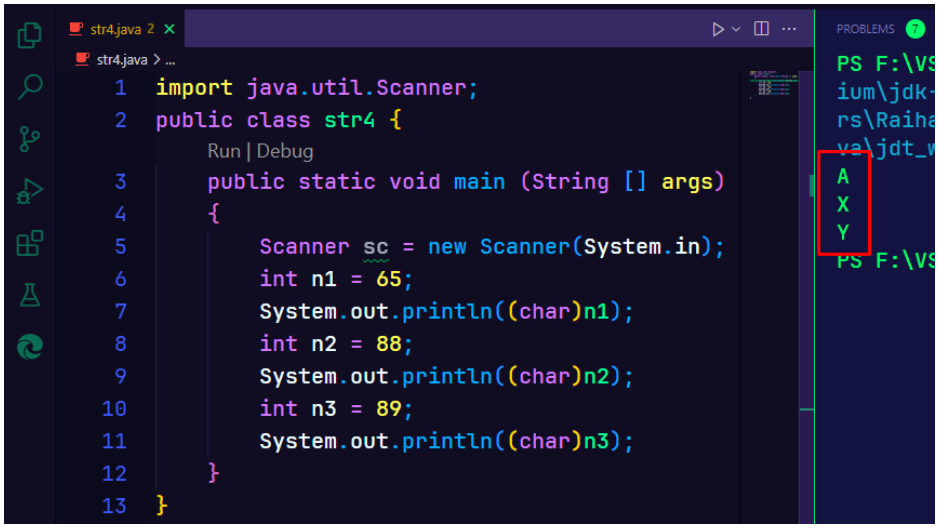
এখানে যেই String নিয়েছি loop এর মাধ্যমে সেটার index অনুযায়ী character এবং সেই character গুলোর ASCII value গুলো বের করে এনেছি।

- int(chr)
  - Can simply cast a character as an integer.



এটাকে casting বলে। মূলত character গুলোর ASCII value বের করে আনা।

আবার উল্টা কাজও করতে পারি অর্থাৎ number/ASCII value থাকলে তার character বের করে আনা।

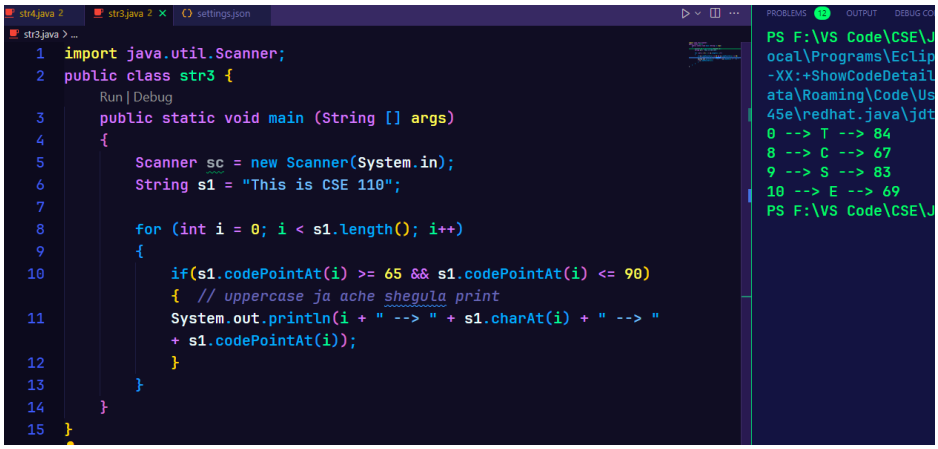




```java
import java.util.Scanner;
public class str3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "This is CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            if(s1.codePointAt(i) >= 97 && s1.codePointAt(i) <= 122)
            {  // Lowercase ja ache shegula print
            System.out.println(i + " --> " + s1.charAt(i) + " --> "
            + s1.codePointAt(i));
            }
        }
    }
}
```

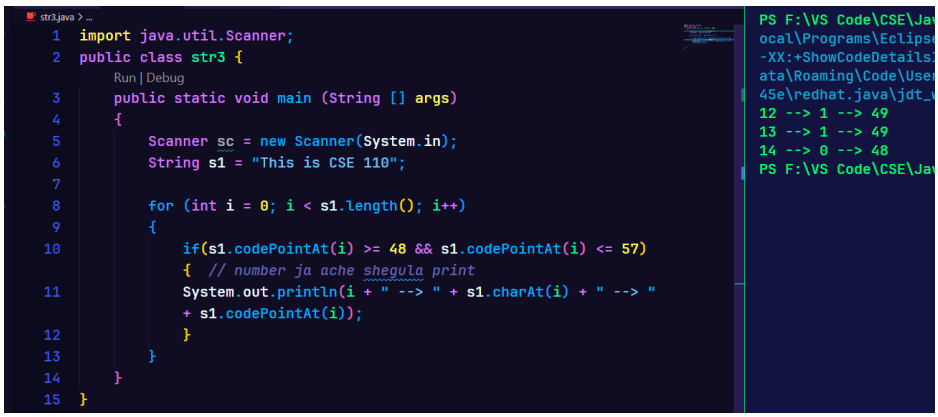
```
PS F:\VS Code\CSE\Java\CSE11
ocal\Programs\Eclipse Adopti
-XX:+ShowCodeDetailsInExcep
ata\Roaming\Code\User\works
45e\redhat.java\jdt_ws\Stri
1 --> h --> 104
2 --> i --> 105
3 --> s --> 115
5 --> i --> 105
6 --> s --> 115
PS F:\VS Code\CSE\Java\CSE11
```



```java
import java.util.Scanner;
public class str3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "This is CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            if(s1.codePointAt(i) >= 65 && s1.codePointAt(i) <= 90)
            {  // uppercase ja ache shegula print
            System.out.println(i + " --> " + s1.charAt(i) + " --> "
            + s1.codePointAt(i));
            }
        }
    }
}
```

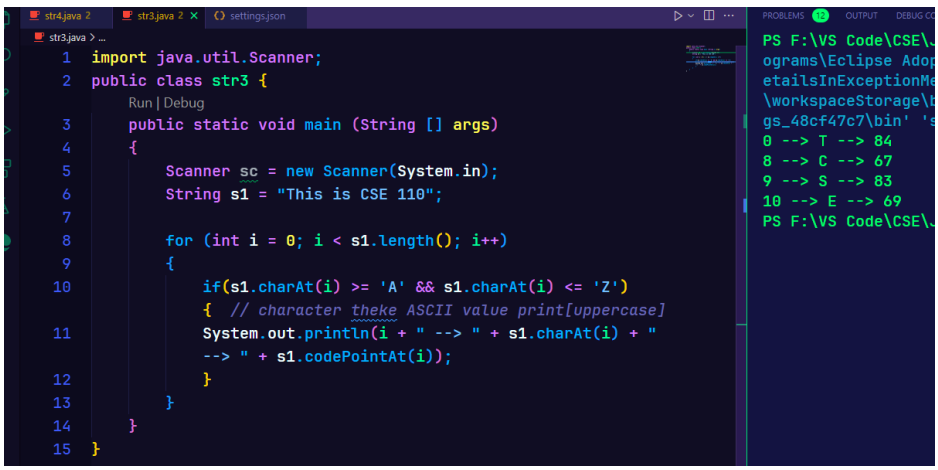
```
PS F:\VS Code\CSE\J
ocal\Programs\Eclip
-XX:+ShowCodeDetail
ata\Roaming\Code\Us
45e\redhat.java\jdt
0 --> T --> 84
8 --> C --> 67
9 --> S --> 83
10 --> E --> 69
PS F:\VS Code\CSE\J
```



```java
import java.util.Scanner;
public class str3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "This is CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            if(s1.codePointAt(i) >= 48 && s1.codePointAt(i) <= 57)
            {  // number ja ache shegula print
            System.out.println(i + " --> " + s1.charAt(i) + " --> "
            + s1.codePointAt(i));
            }
        }
    }
}
```

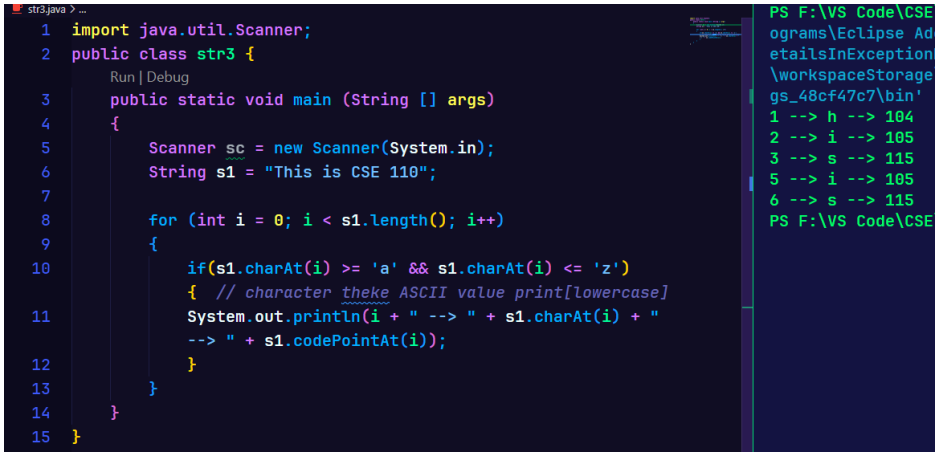
```
PS F:\VS Code\CSE\Jav
ocal\Programs\Eclipse
-XX:+ShowCodeDetails
ata\Roaming\Code\User
45e\redhat.java\jdt_
12 --> 1 --> 49
13 --> 1 --> 49
14 --> 0 --> 48
PS F:\VS Code\CSE\Jav
```



```java
import java.util.Scanner;
public class str3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "This is CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            if(s1.charAt(i) >= 'A' && s1.charAt(i) <= 'Z')
            {  // character theke ASCII value print[uppercase]
            System.out.println(i + " --> " + s1.charAt(i) + "
            --> " + s1.codePointAt(i));
            }
        }
    }
}
```

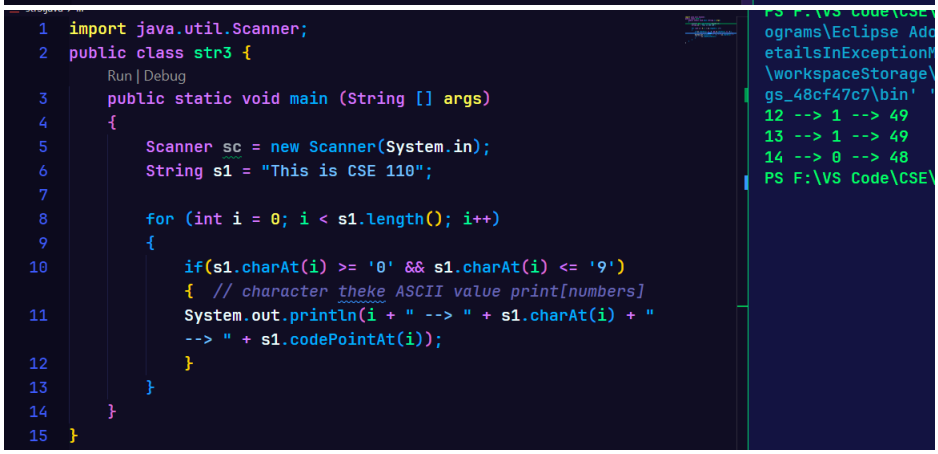
```
PS F:\VS Code\CSE\
ograms\Eclipse Adop
etailsInExceptionMe
\workspaceStorage\b
gs_48cf47c7\bin' 's
0 --> T --> 84
8 --> C --> 67
9 --> S --> 83
10 --> E --> 69
PS F:\VS Code\CSE\
```



```java
import java.util.Scanner;
public class str3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "This is CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            if(s1.charAt(i) >= 'a' && s1.charAt(i) <= 'z')
            {  // character theke ASCII value print[lowercase]
            System.out.println(i + " --> " + s1.charAt(i) + "
            --> " + s1.codePointAt(i));
            }
        }
    }
}
```

```
PS F:\VS Code\CSE
ograms\Eclipse Ad
etailsInException
\workspaceStorage
gs_48cf47c7\bin'
1 --> h --> 104
2 --> i --> 105
3 --> s --> 115
5 --> i --> 105
6 --> s --> 115
PS F:\VS Code\CSE
```



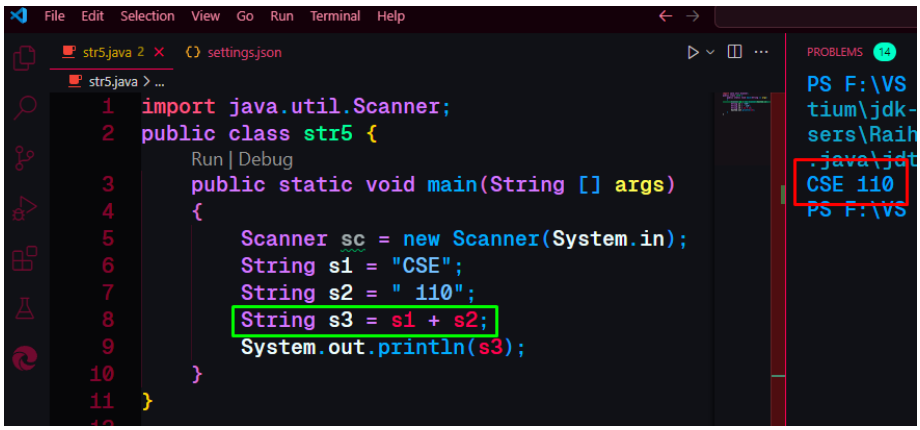
```java
import java.util.Scanner;
public class str3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "This is CSE 110";

        for (int i = 0; i < s1.length(); i++)
        {
            if(s1.charAt(i) >= '0' && s1.charAt(i) <= '9')
            {  // character theke ASCII value print[numbers]
            System.out.println(i + " --> " + s1.charAt(i) + "
            --> " + s1.codePointAt(i));
            }
        }
    }
}
```

```
ograms\Eclipse Ad
etailsInExceptionM
\workspaceStorage\
gs_48cf47c7\bin'
12 --> 1 --> 49
13 --> 1 --> 49
14 --> 0 --> 48
PS F:\VS Code\CSE\
```

String Concatenation

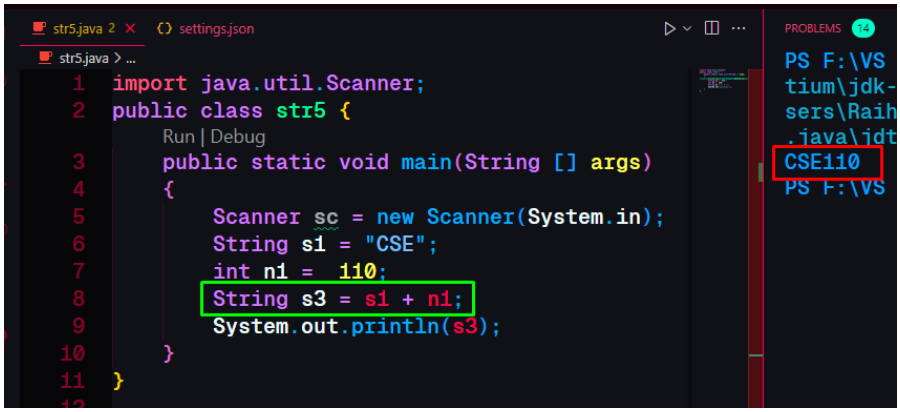
- “+” is used to concatenate two or more strings together.
- New string created after concatenation.
- If a different data type follows a string data type in concatenation, that data is automatically converted to string.



```java
import java.util.Scanner;
public class str5 {
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = "CSE";
        String s2 = " 110";
        String s3 = s1 + s2;
        System.out.println(s3);
    }
}
```

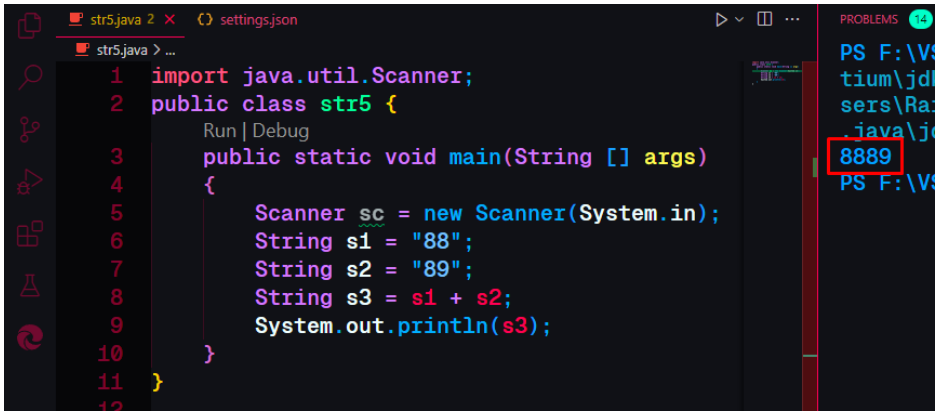
```
PS F:\VS
tium\jdk-
sers\Raih
CSE 110
PS F:\VS
```

- Number + Number = Addition
- “String” + Number = Concatenation



- “String” + “String” = String

অর্থাৎ, ২ টা জিনিস (“ ”) এর ভিতরে/String হিসেবে থাকলে Attach হয়ে যাবে।



Comparing Strings

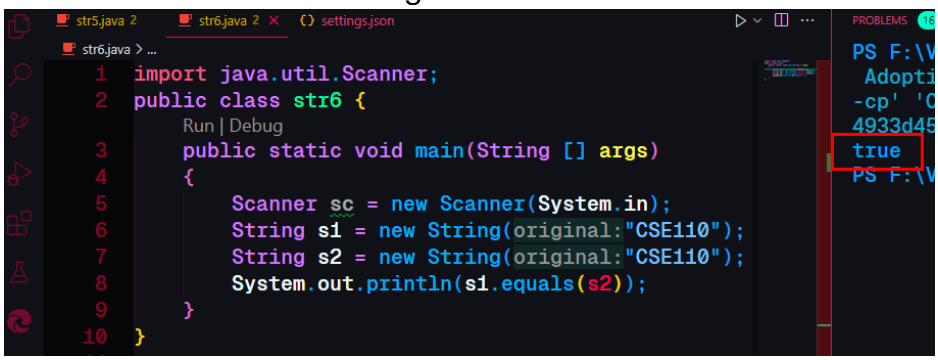
- String str1 = new String(“Hello”); String str2 = new String(“hello”);

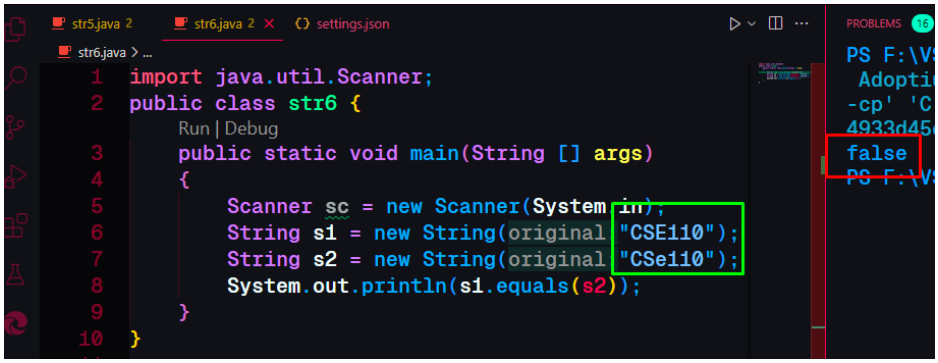
Methods	Functions	Method Call	Value returned
.equals( )	Checks if values in two strings are same. Case sensitive.	str1.equals(str2)	False
.equalsIgnoreCase( )	Checks if values in two strings are same. Not case sensitive.	str1.equalsIgnoreCase(str2)	True

== or equals operator	Checks if reference (address) of two strings are same.	str1 == str2	False
.compareTo( )	Compares two string values lexicographically. Returns 0 when identical, (+)ve when first string is greater and (-)ve values when second string is greater respectively.	str1.compareTo(str2)	-32 (The output is -32 since “H” is 32 less than “h” lexicograp hically)

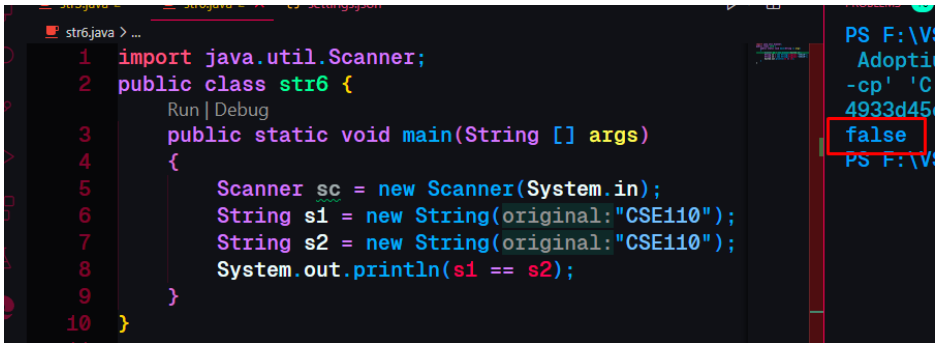
.equals( )

Checks if values in two strings are same. Case sensitive.





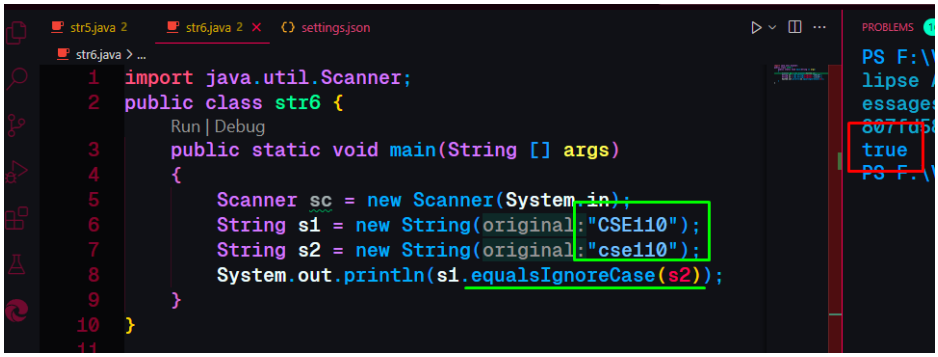
== or equals operator



== সবসময় Reference(Address) যাচাই করে।

.equalsIgnoreCase( )

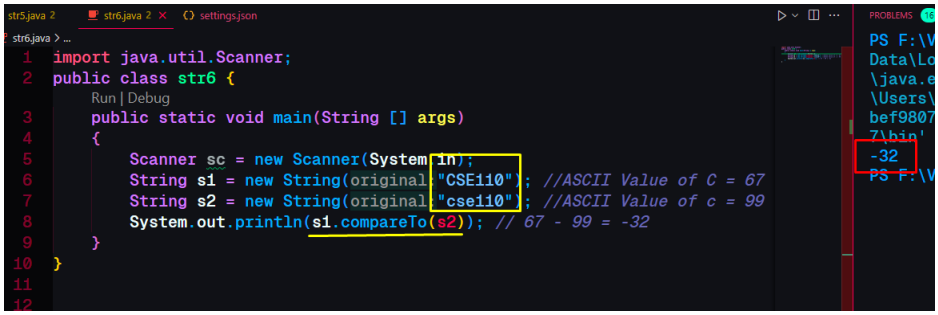
Checks if values in two strings are same. Not case sensitive.



.compareTo( )

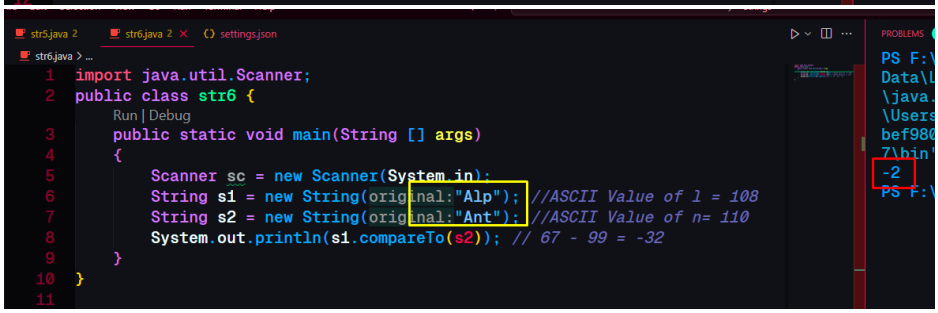
Compares two string values lexicographically. Returns 0 when identical, (+)ve when

first string is greater and (-)ve values when second string is greater respectively.



```java
import java.util.Scanner;
public class str6 {
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = new String(original:"CSE110"); //ASCII Value of C = 67
        String s2 = new String(original:"cse110"); //ASCII Value of c = 99
        System.out.println(s1.compareTo(s2)); // 67 - 99 = -32
    }
}
```

```
PS F:\V
Data\Lo
\java.e
\Users\
bef9807
7\bin'
-32
PS F:\V
```



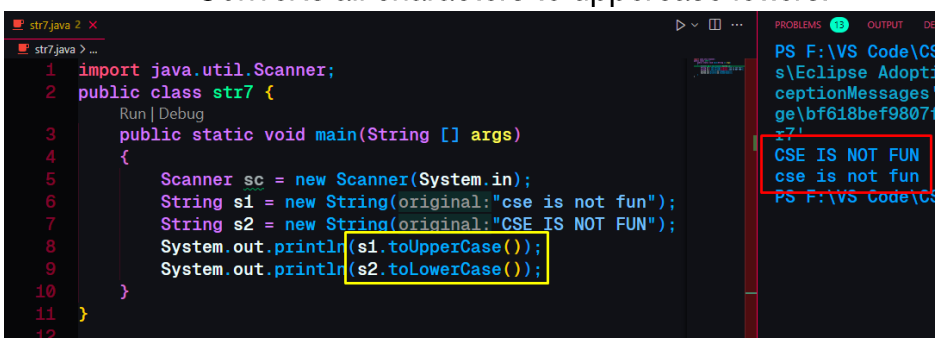
```java
import java.util.Scanner;
public class str6 {
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = new String(original:"Alp"); //ASCII Value of l = 108
        String s2 = new String(original:"Ant"); //ASCII Value of n= 110
        System.out.println(s1.compareTo(s2)); // 67 - 99 = -32
    }
}
```

```
PS F:\
Data\L
\java.
\Users
bef980
7\bin'
-2
PS F:\
```

প্রথম অমিল যেখানে পাচ্ছে সেখানে পরস্পর ২ টি character এর ASCII value compare বা পার্থক্য বের করছে।

Case Conversion

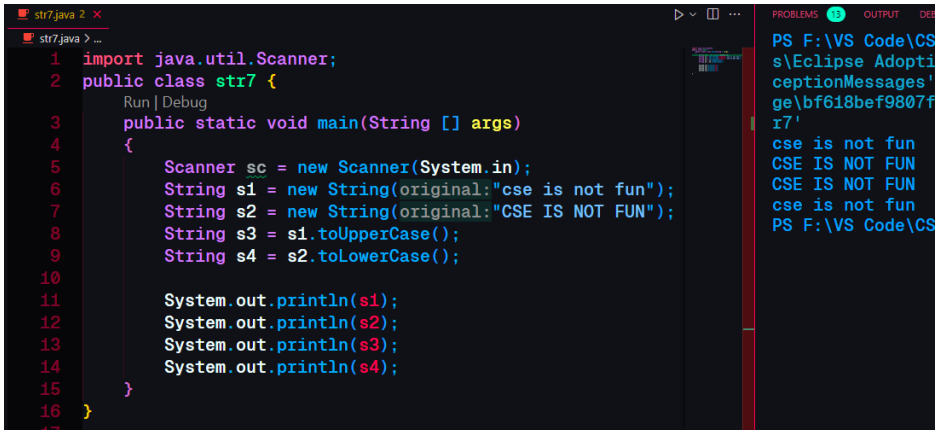
- .toLowerCase( )
  - Converts all characters to lowercase letters.
- .toUpperCase( )
  - Converts all characters to uppercase letters.



```java
import java.util.Scanner;
public class str7 {
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        String s1 = new String(original:"cse is not fun");
        String s2 = new String(original:"CSE IS NOT FUN");
        System.out.println(s1.toUpperCase());
        System.out.println(s2.toLowerCase());
    }
}
```

```
PS F:\VS Code\CS
s\Eclipse Adopt
ceptionMessages
ge\bf618bef9807
str7'
CSE IS NOT FUN
cse is not fun
PS F:\VS Code\CS
```

String কে কখনোই আমরা change করতে পারি না। String কে change করতে হলে আবার নতুন করে String তৈরি করতে হবে।



এখানে, s1 & s2 পরিবর্তন হয়নি। s1, s2 এর ১ টা UpperCase & LowerCase version তৈরি করে s3 & s4 এ store করে দিয়েছে।

CW

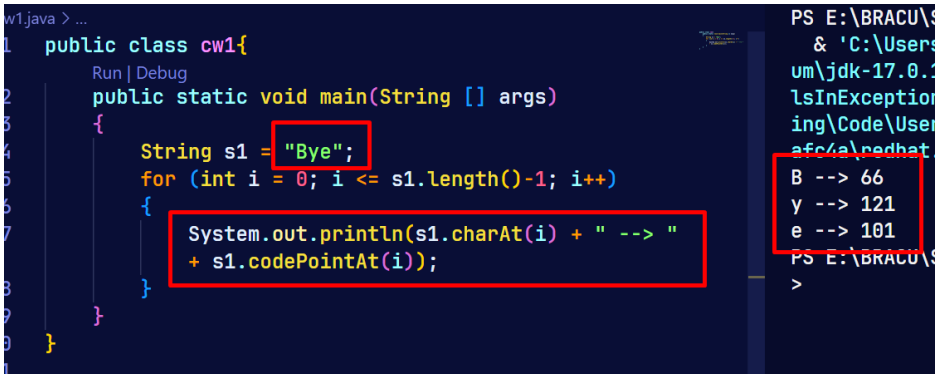
1. You have given a String. Print characters & their ASCII values.



```
public class cw1{
    Run | Debug
    public static void main(String [] args)
    {
        String s1 = "Bye";
        for (int i = 0; i <= s1.length()-1; i++)
        {
            char ch = s1.charAt(i);
            int ascii = (int)ch;
            System.out.println(ch + " --> " + ascii);
        }
    }
}
```

```
B --> 66
y --> 121
e --> 101
```

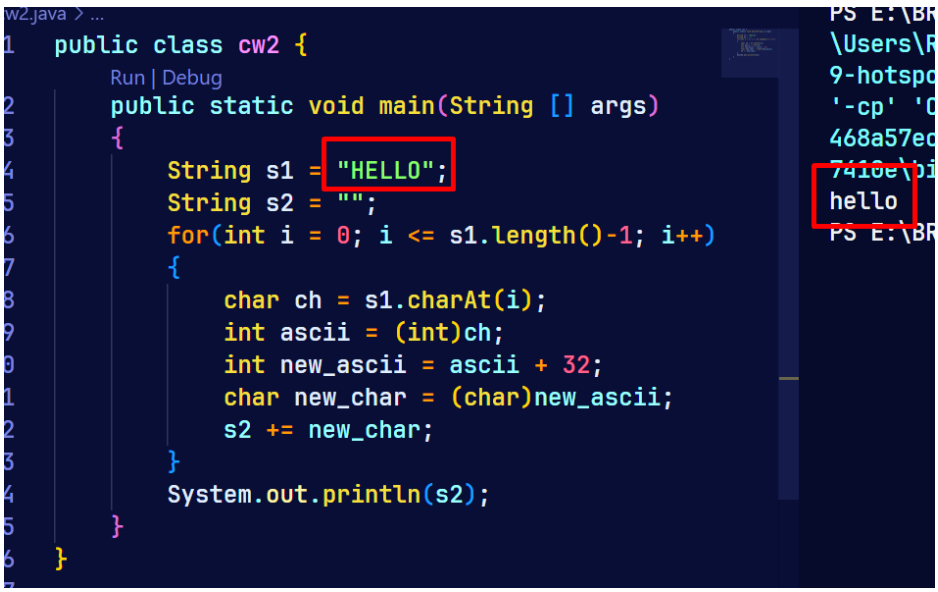
অন্যভাবে,



```
public class cw1{
    Run | Debug
    public static void main(String [] args)
    {
        String s1 = "Bye";
        for (int i = 0; i <= s1.length()-1; i++)
        {
            System.out.println(s1.charAt(i) + " --> "
            + s1.codePointAt(i));
        }
    }
}
```

```
B --> 66
y --> 121
e --> 101
```

2. Write a Java Program where a given String which has all Uppercase. Convert them to Lowercase.



```
public class cw2 {
    Run | Debug
    public static void main(String [] args)
    {
        String s1 = "HELLO";
        String s2 = "";
        for(int i = 0; i <= s1.length()-1; i++)
        {
            char ch = s1.charAt(i);
            int ascii = (int)ch;
            int new_ascii = ascii + 32;
            char new_char = (char)new_ascii;
            s2 += new_char;
        }
        System.out.println(s2);
    }
}
```

```
hello
```

Arrays

What is an array?

- An array is a collection of values of the same data type.
- The length of an array remains constant once it's created.
- Multiple elements are separated using comma when initialized.
- Arrays are considered as objects in java.
- Number of elements of an array is called the length of an array.

◆ **An Array is an Indexed collection of, fixed number of Homogeneous Data elements.**

◆ **Homogeneous** বলতে একই রকম Data type বোঝায়। যদি int [ ] array নিয়ে value, array[2] = 4.44 অর্থাৎ double value নেয়া হয় তাহলে error দিবে।

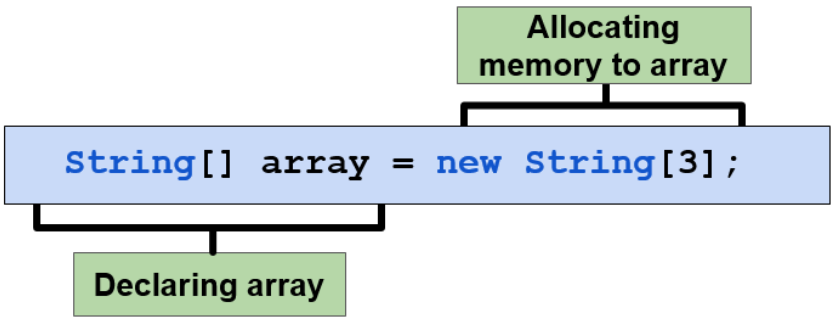
◆ We can represent huge number of values by using single variable so that Readability of the code will be improved. (একটা Variable এ অনেক বেশি value নিতে পারি যার ফলে Code পড়তে/বুঝতে সুবিধা হয়।)

◆ But the main disadvantage of arrays is fixed in size. Once we create an array, there is no chance of increasing/decreasing the size based on our requirement. (প্রধান অসুবিধা হলো Array এর সাইজ কিংবা আকার সুনির্দিষ্ট। একবার ১ টি Array তৈরি করা হয়ে গেলে কোনোভাবেই আর আমাদের প্রয়োজন অনুসারে Array এর আয়তন বাড়াতে/কমাতে পারবো না।)

◆ Hence, To use arrays concept, compulsory we should know the size in advance, which may not possible always. (Array এর ব্যাপারগুলো ব্যবহার করতে হলে Array এর আয়তন আগে থেকেই বাধ্যতামূলকভাবে জানা উচিত, যেটা সবসময় সম্ভব নাও হতে পারে।)

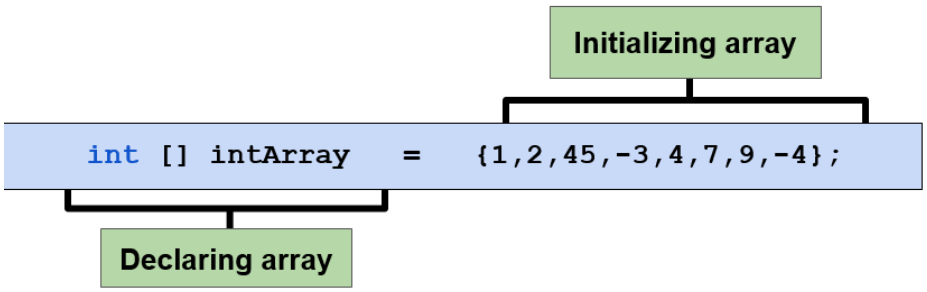
Array Declaration

- “new” Keyword is used to create arrays.
- While initializing an array we declare the type of the elements and the size of the array
- Initially array contains the default values of the specified data type.
- Syntax:



Array Initialization

- We can initialize an array at the time of declaration.
- Initializing using an array initializer (a comma-separated list of expressions enclosed in braces)
- Syntax:



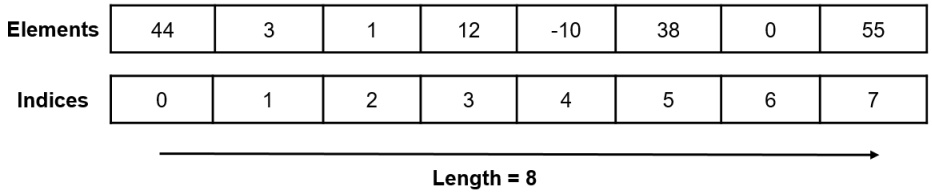

```java
import java.util.Arrays;
import java.util.Scanner;
public class arr1{
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        int [] arr1 = new int [4];
        int [] arr2 = new int [] {11, 12, 13, 14};
        int [] arr3 = {11, 12, 13, 14};
        System.out.println(Arrays.toString(arr1));
        System.out.println(Arrays.toString(arr2));
        System.out.println(Arrays.toString(arr3));
    }
}
```

```
PS F:\VS Code\CSE\
e Adoptium\jdk-17.
' '-cp' 'C:\Users\
38d202d84d8b6\redh
[0, 0, 0, 0]
[11, 12, 13, 14]
[11, 12, 13, 14]
PS F:\VS Code\CSE\
```

Index of an Array

- Each elements of an array holds a position or index.

- The first element in every array has index **zero**.
- The last element has index number (**array.length - 1)**
- Index number is enclosed by square brackets [ ]




```java
import java.util.Arrays;
import java.util.Scanner;
public class arr1{
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        int [] arr1 = {11, 12, 13, 14};
        System.out.println(arr1.length);
    }
}
```

```
PS F:\V
tium\jd
Users\R
at.java
4
PS F:\V
```

Array এর ক্ষেত্রে method হচ্ছে .length। .length( ) নয়।

```java
import java.util.Arrays;
import java.util.Scanner;
public class arr1{
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        int [] arr1 = {11, 12, 13, 14};
        System.out.println(arr1[2]);
    }
}
```

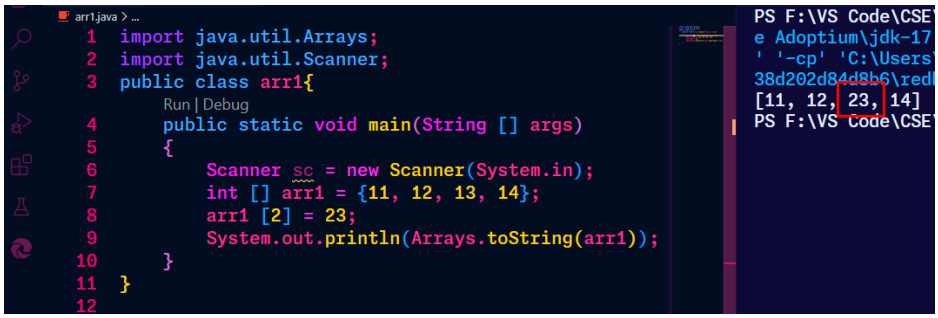
```
PS F:\V
ptium\j
:\Users
odhat.j
13
PS F:\V
```

২ নং index এ যেটা আছে সেটা print করা। এভাবে যেকোনো index এর value print করা যায়।

```java
import java.util.Arrays;
import java.util.Scanner;
public class arr1{
    Run | Debug
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        int [] arr1 = {11, 12, 13, 14};
        System.out.println(arr1[arr1.length-1]);
    }
}
```

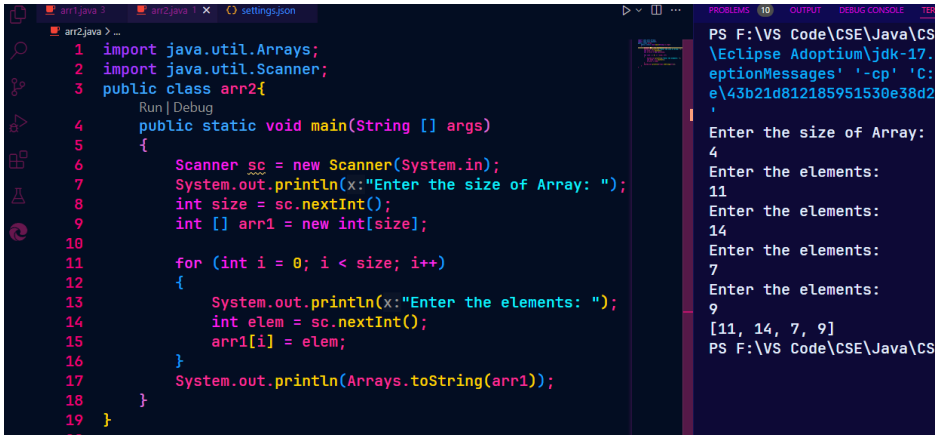
```
PS F:\V
Adoptiu
cp' 'C:
14
PS F:\V
```

Array এর শেষ value print করতে চাইলে index অনুযায়ী arr1[ ] দিয়ে করা যায়। একই সাথে এভাবেও করা যায়।



Array এর value পরিবর্তন করা।

User থেকে input নিয়ে array তৈরি করাঃ



Array Iteration

- Array iteration means accessing each element of an array one by one.
- We can use the concept of loops to iterate an array.

Array Iteration - while loop

```java
import java.util.Arrays;
import java.util.Scanner;

public class arr3 {
    Run | Debug
    public static void main (String [] args)
    {
        Scanner sc = new Scanner(System.in);
        int [] arr1 = {11, 12, 13, 14};
        System.out.println(x:"Elements of the Array are: ");

        int i = 0;
        while(i < arr1.length)
        {
            System.out.println(arr1[i]);
            i++;
        }
    }
}
```

```
PS F:\VS Code\CSE\Java\CSE1
ams\Eclipse Adoptium\jdk-17
sInExceptionMessages' '-cp'
aceStorage\43b21d8121859515
5\bin' 'arr3'
Elements of the Array are:
11
12
13
14
PS F:\VS Code\CSE\Java\CSE1
```

Array Iteration - for loop

```java
import java.util.Arrays;
import java.util.Scanner;

public class arr4 {
    Run | Debug
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int[] arr1 = { 11, 12, 13, 14 };
        System.out.println(x:"Elements of the Array are: ");
        for (int i = 0; i < arr1.length; i++)
        {
            System.out.println(arr1[i]);
        }
    }
}
```

```
PS F:\VS Code\CSE\Java\CSE1
ams\Eclipse Adoptium\jdk-17
sInExceptionMessages' '-cp'
aceStorage\43b21d8121859515
5\bin' 'arr4'
Elements of the Array are:
11
12
13
14
PS F:\VS Code\CSE\Java\CSE1
```

একই array কে উল্টা (বিপরীত দিক থেকে) print করতে চাইলেঃ

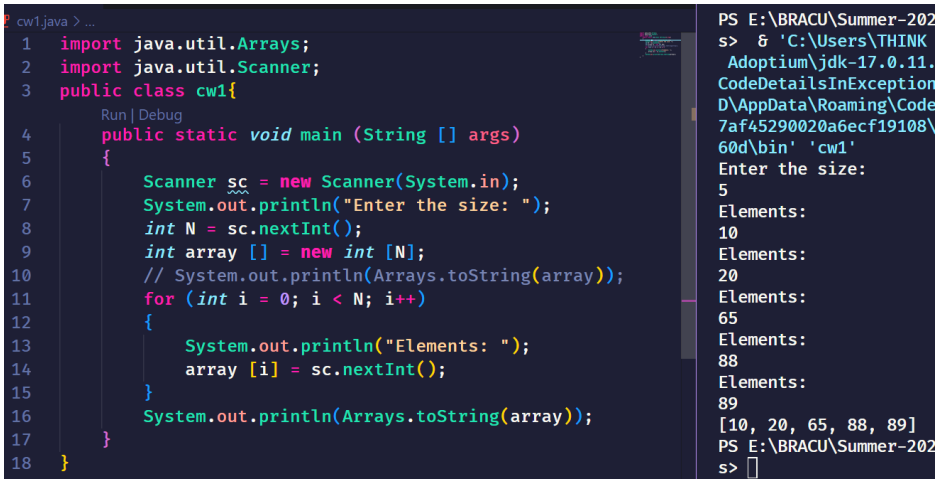
```java
import java.util.Arrays;
import java.util.Scanner;

public class arr5 {
    Run | Debug
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int[] arr1 = { 11, 12, 13, 14 };
        System.out.println(x:"Elements of the Array are: ");
        for (int i = arr1.length-1; i >= 0; i--)
        {
            System.out.println(arr1[i]);
        }
    }
}
```

```
PS F:\VS Code\CSE\Java\CSE1
ms\Eclipse Adoptium\jdk-17.
nExceptionMessages' '-cp' '
Storage\43b21d812185951530e
n' 'arr5'
Elements of the Array are:
14
13
12
11
PS F:\VS Code\CSE\Java\CSE1
```

CW

Take the size N as input from the user and create an integer array which stores N elements inside the array.

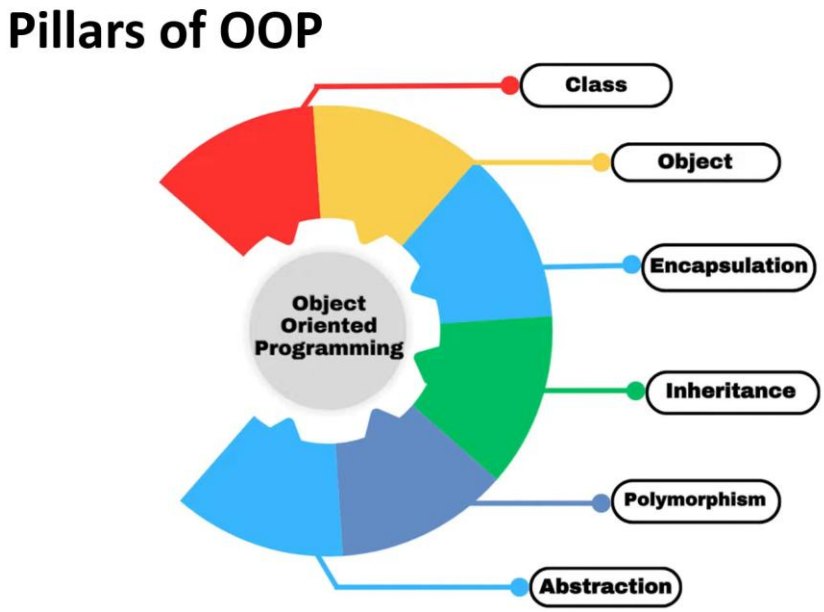


```
cw1.java > ...
1   import java.util.Arrays;
2   import java.util.Scanner;
3   public class cw1{
        Run | Debug
4       public static void main (String [] args)
5       {
6           Scanner sc = new Scanner(System.in);
7           System.out.println("Enter the size: ");
8           int N = sc.nextInt();
9           int array [] = new int [N];
10          // System.out.println(Arrays.toString(array));
11          for (int i = 0; i < N; i++)
12          {
13              System.out.println("Elements: ");
14              array [i] = sc.nextInt();
15          }
16          System.out.println(Arrays.toString(array));
17      }
18  }
```

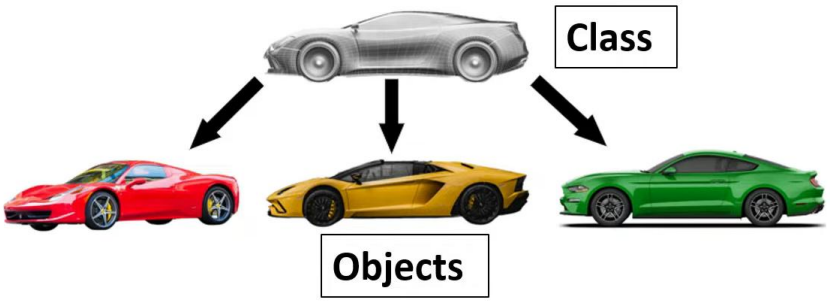
```
PS E:\BRACU\Summer-202
s>  & 'C:\Users\THINK
 Adoptium\jdk-17.0.11.
CodeDetailsInException
D\AppData\Roaming\Code
7af45290020a6ecf19108\
60d\bin' 'cw1'
Enter the size:
5
Elements:
10
Elements:
20
Elements:
65
Elements:
88
Elements:
89
[10, 20, 65, 88, 89]
PS E:\BRACU\Summer-202
s>
```

Resize the array by 1 and add one more element.

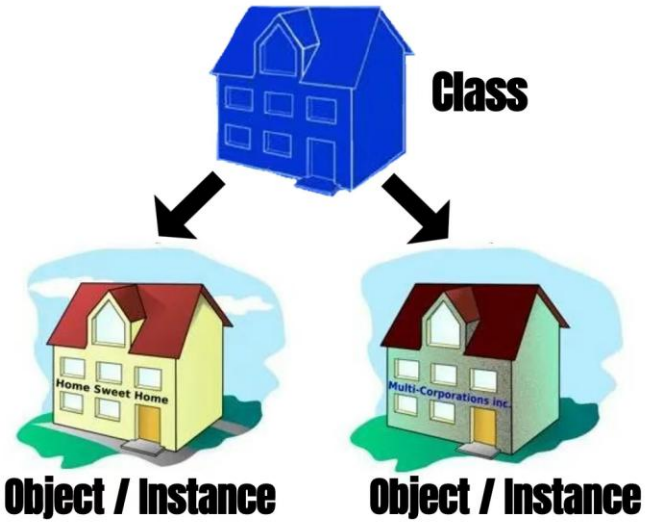
Intro to OOP



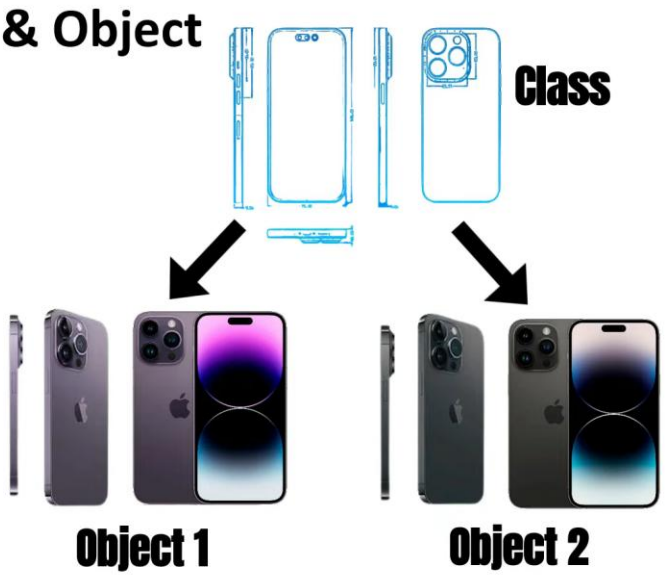
- Class → Template/Design/Map/Blueprint
- Object/Instance → Real life entities of class.



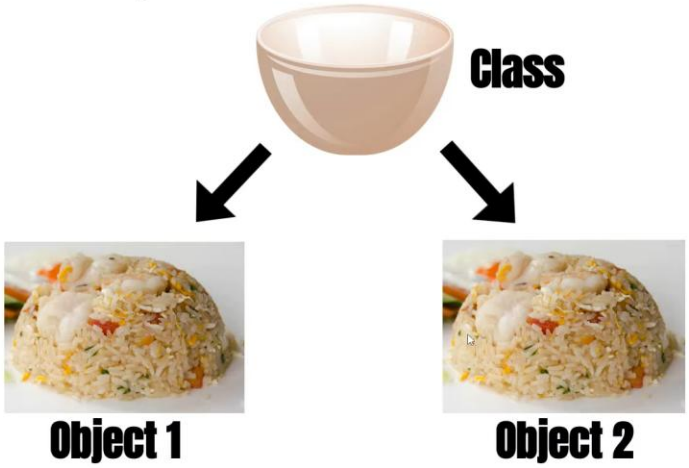
Class & Object



Class & Object



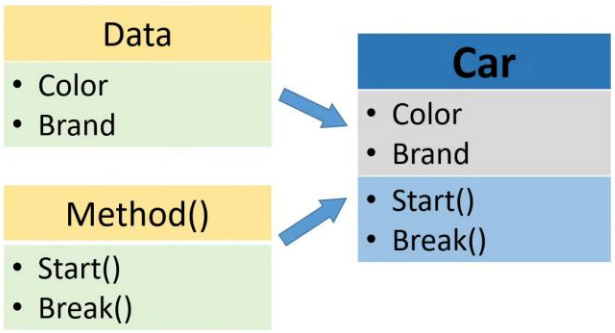
Class & Object



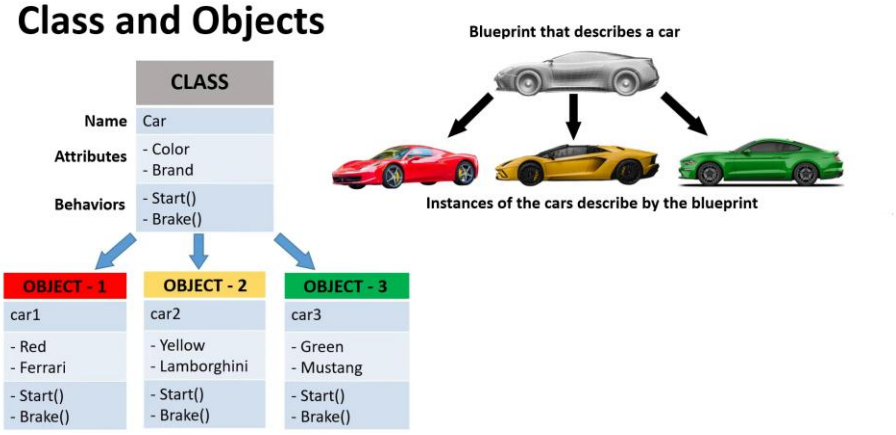
Class Components

- Fields (the data/attributes)
- Behavior (the methods)





Class and Objects

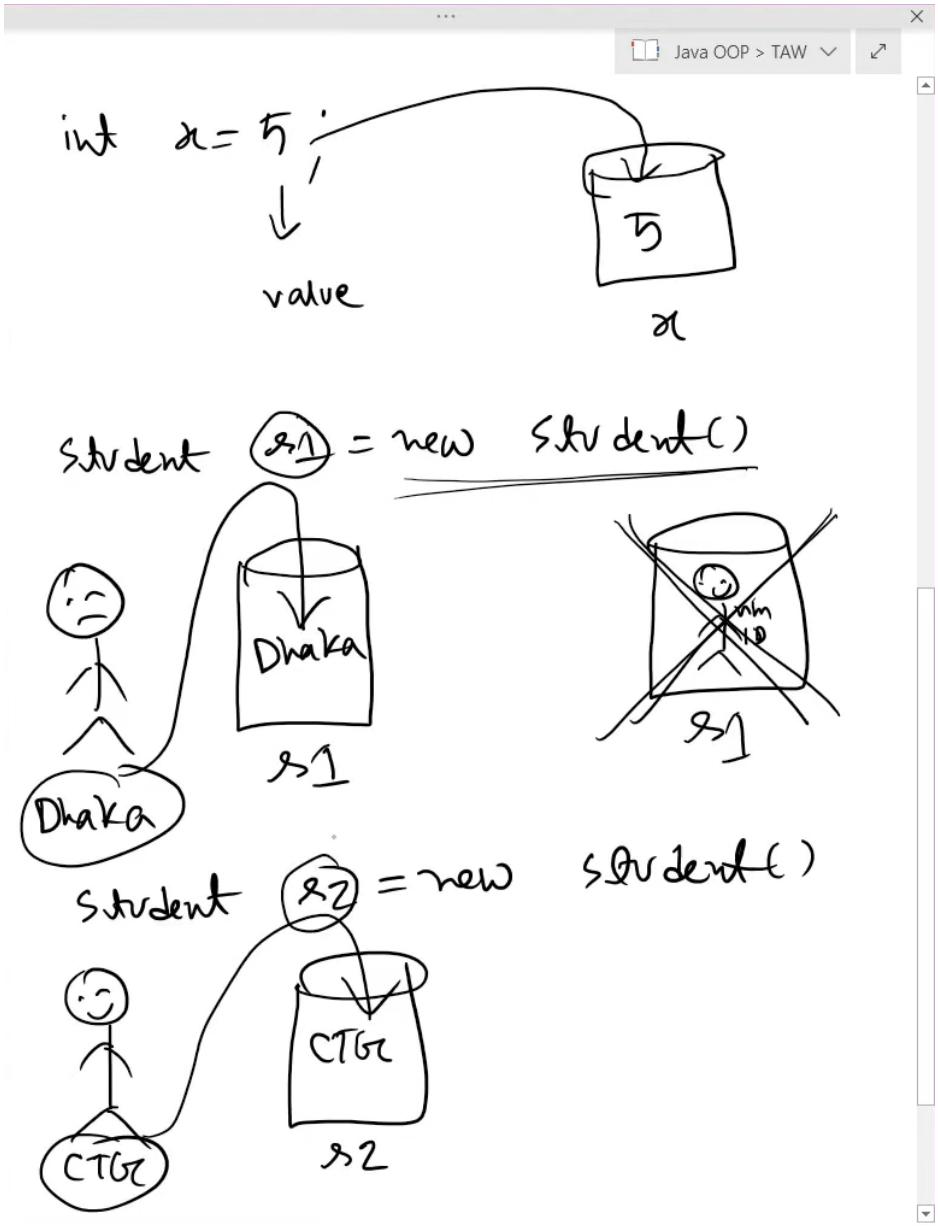


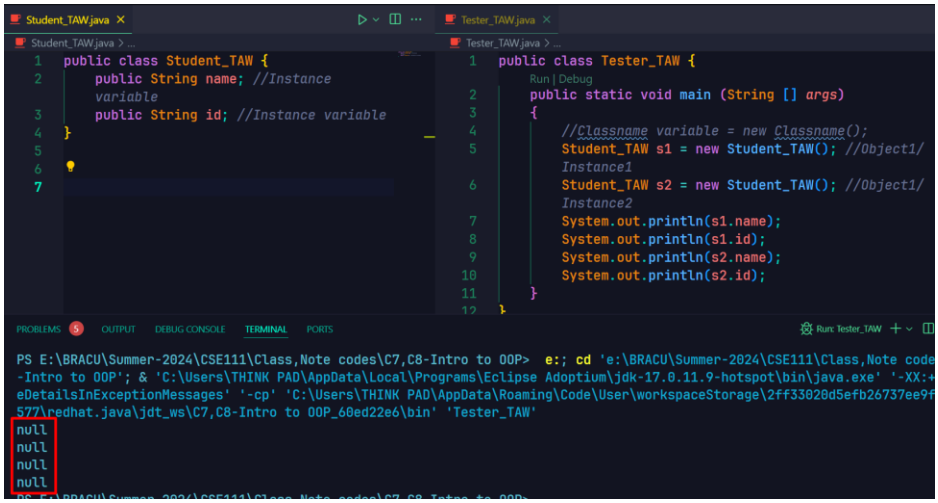
```java
public class Student_TAW {
    public String name;
    public String id;
}
```

```java
public class Tester_TAW {
    public static void main (String [] args)
    {
        //Classname variable = new Classname();
        Student_TAW s1 = new Student_TAW(); //Object1/Instance1
        Student_TAW s2 = new Student_TAW(); //Object1/Instance2
        int x = 5;
        System.out.println(x);
        System.out.println(s1);
        System.out.println(s2);
    }
}
```

```
PS E:\BRACU\Summer-2024\CSE111\Class,Note codes\C7,C8-Intro to OOP>  & 'C:\Users\THINK PAD\AppData\Local\Programs\Eclipse Adoptium\jdk-17.0.11.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\THINK PAD\AppData\Roaming\Code\User\workspaceStorage\2ff33020d5efb26737ee9f62c57dc577\redhat.java\jdt_ws\C7,C8-Intro to OOP_60ed22e6\bin' 'Tester_TAW'
5
Student_TAW@5acf9800
Student_TAW@4617c264
```

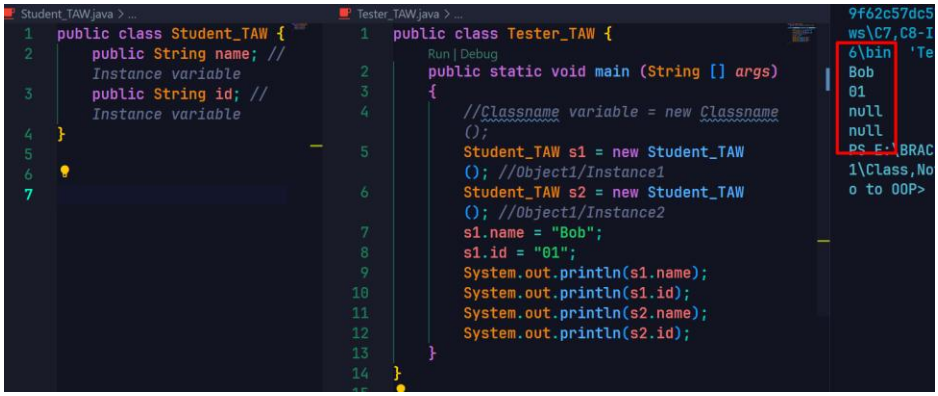
s1, s2 এর ক্ষেত্রে memory location print করছে।

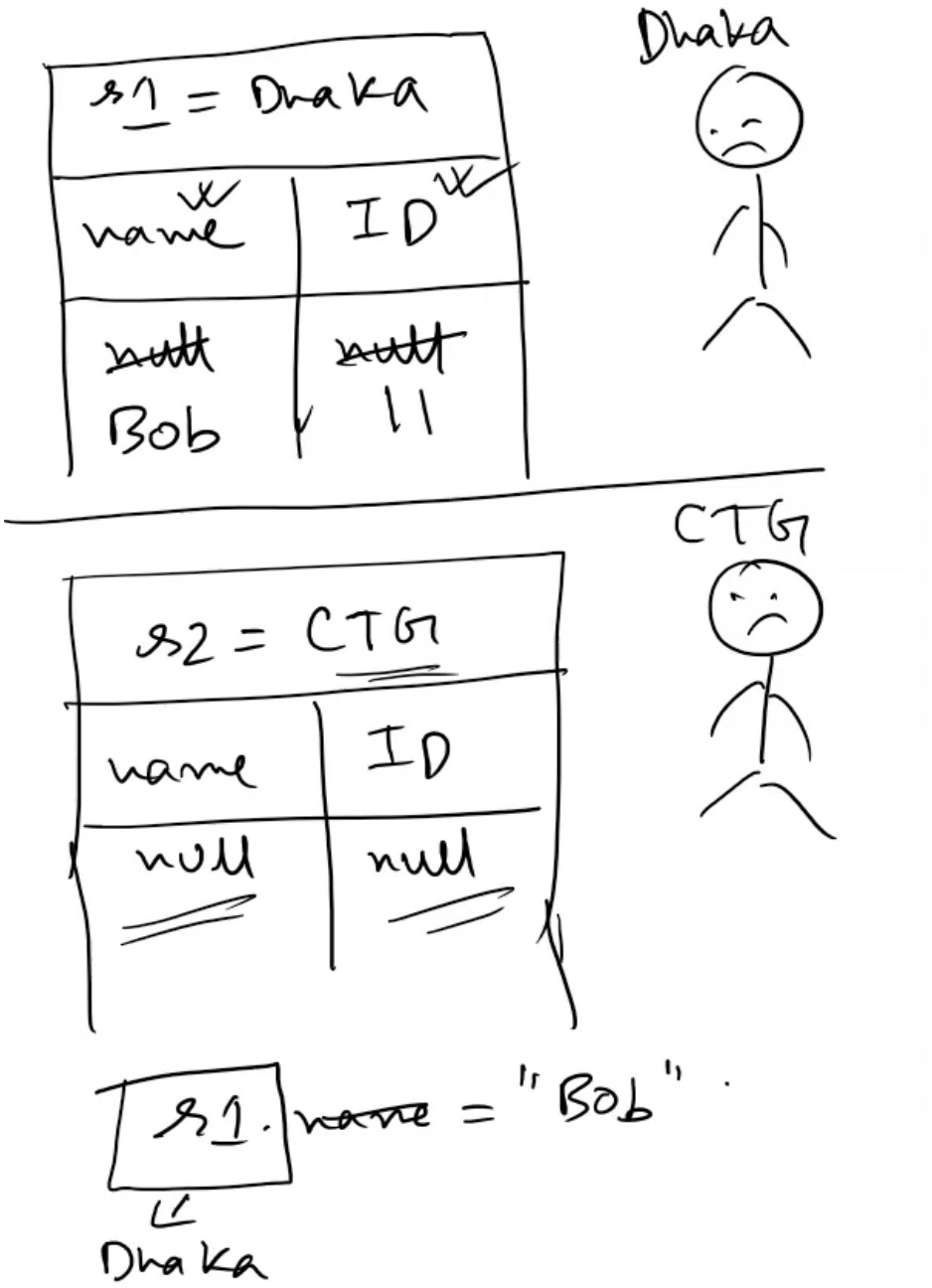




◆ Instance Variable কে access করতে হলে Object এর Reference দরকার হয়।

Datatype String হওয়ায় Default value = null দেখাচ্ছে।



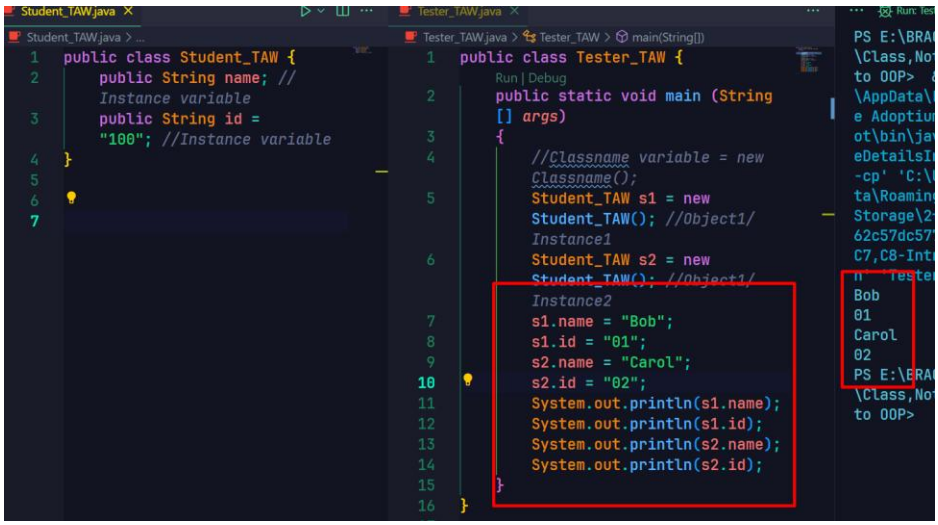



```java
public class Student_TAW {
    public String name; //
    Instance variable
    public String id =
    "100"; //Instance variable
}
```

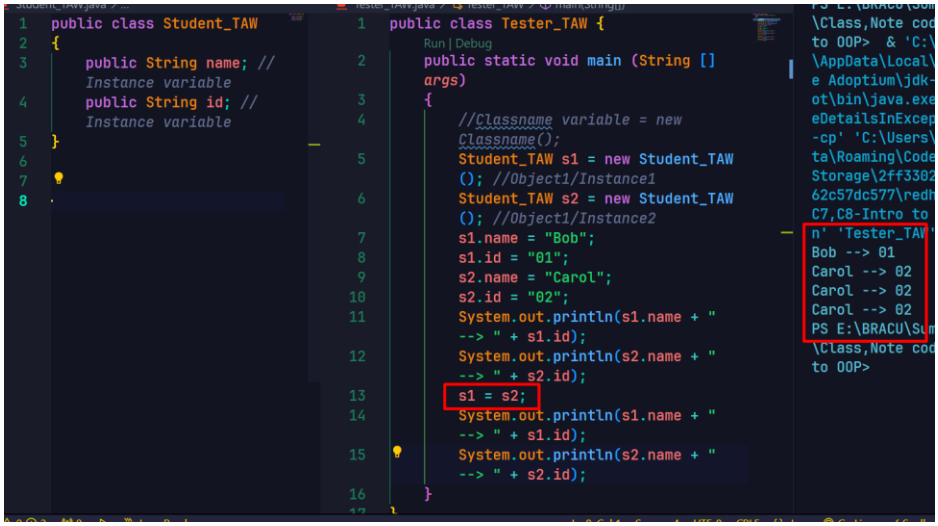
```java
public class Tester_TAW {
    Run | Debug
    public static void main (String
    [] args)
    {
        //Classname variable = new
        Classname();
        Student_TAW s1 = new
        Student_TAW(); //Object1/
        Instance1
        Student_TAW s2 = new
        Student_TAW(); //Object1/
        Instance2
        s1.name = "Bob";
        //s1.id = "01";
        System.out.println(s1.name);
        System.out.println(s1.id);
        System.out.println(s2.name);
        System.out.println(s2.id);
    }
}
```

```
AW'
Bob
100
null
100
PS E:\BRA
Class,Not
 OOP>
```

Design Class এ বলা আছে যখনই কোনো Student create হচ্ছে সেই Student এর id হবে 100.



value update

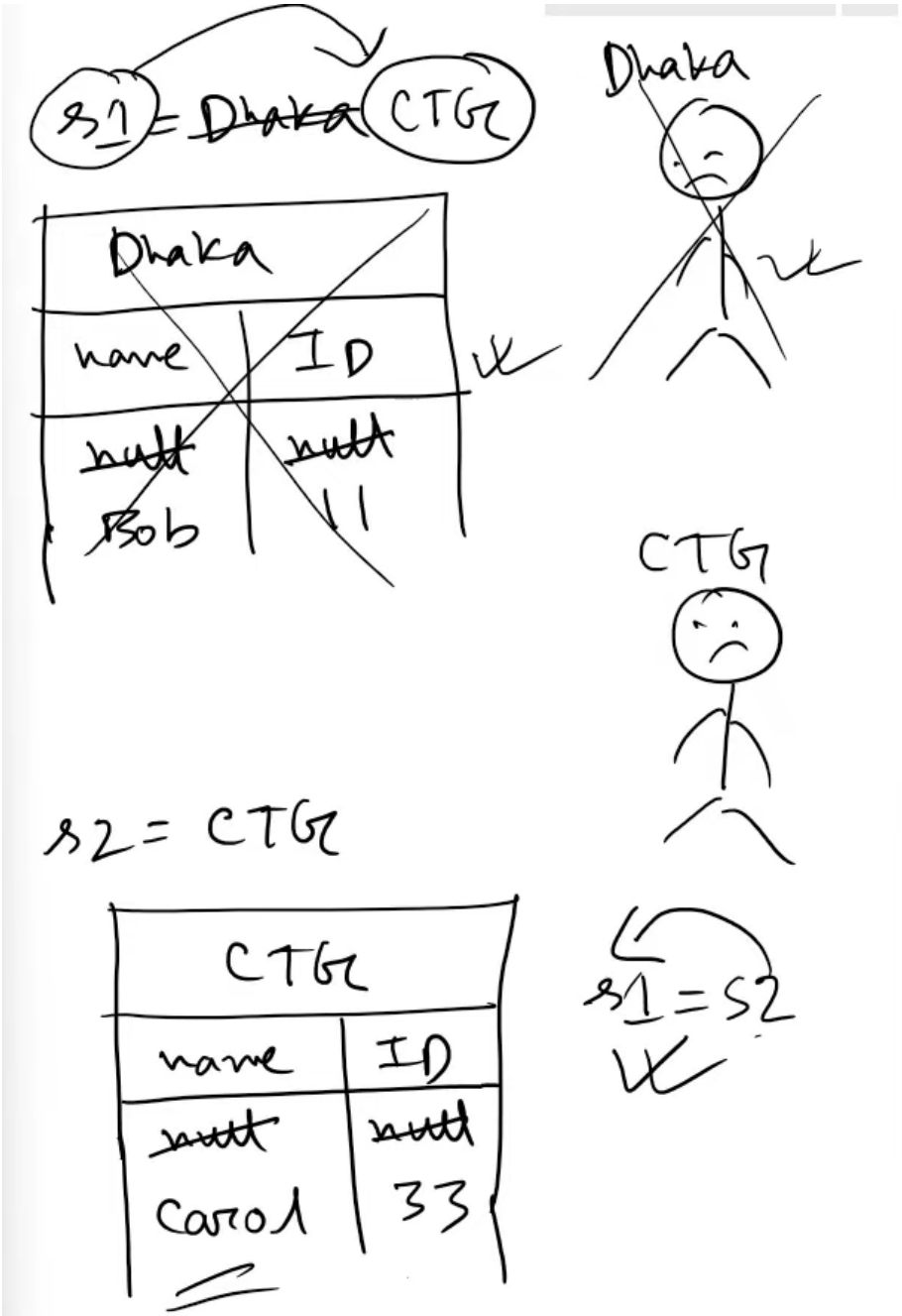


```
public class Student_TAW
{
    public String name; //
    Instance variable
    public String id; //
    Instance variable
}
```

```
public class Tester_TAW {
    Run | Debug
    public static void main (String []
    args)
    {
        //Classname variable = new
        Classname();
        Student_TAW s1 = new Student_TAW
        (); //Object1/Instance1
        Student_TAW s2 = new Student_TAW
        (); //Object1/Instance2
        s1.name = "Bob";
        s1.id = "01";
        s2.name = "Carol";
        s2.id = "02";
        System.out.println(s1.name + "
        --> " + s1.id);
        System.out.println(s2.name + "
        --> " + s2.id);
        s1 = s2;
        System.out.println(s1.name + "
        --> " + s1.id);
        System.out.println(s2.name + "
        --> " + s2.id);
    }
```

```
\Class,Note cod
to OOP>  & 'C:\
\AppData\Local\
e Adoptium\jdk-
ot\bin\java.exe
eDetailsInExcep
-cp' 'C:\Users\
ta\Roaming\Code
Storage\2ff3302
62c57dc577\redH
C7,C8-Intro to
n' 'Tester_TAW'
Bob --> 01
Carol --> 02
Carol --> 02
Carol --> 02
PS E:\BRACU\Sum
\Class,Note cod
to OOP>
```

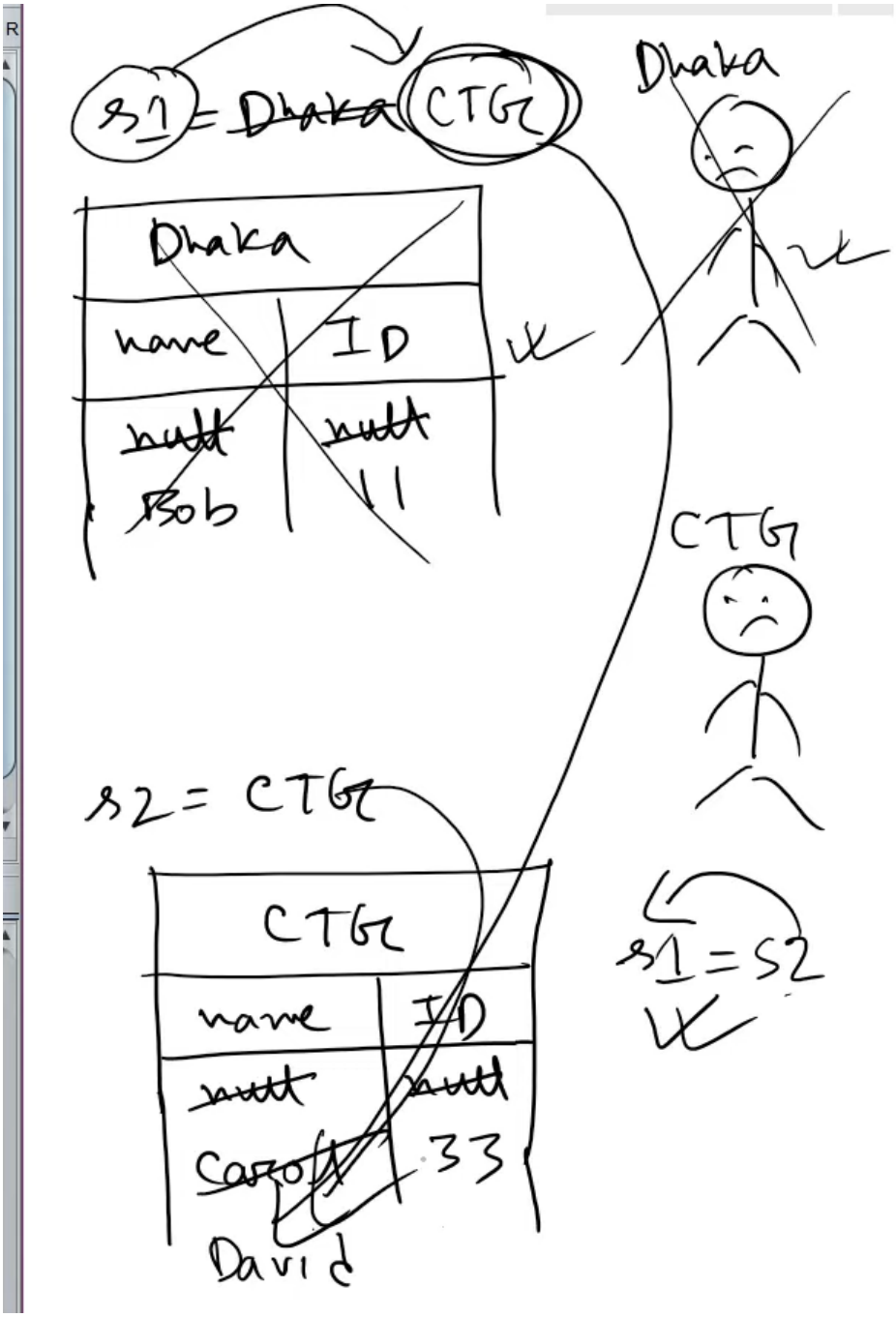
◆ Coding এ সবসময় ডান পাশের value বাম পাশে এসে বসে। যেমনঃ x = 5 হলে, 5 গিয়ে x এ জমা হয়।

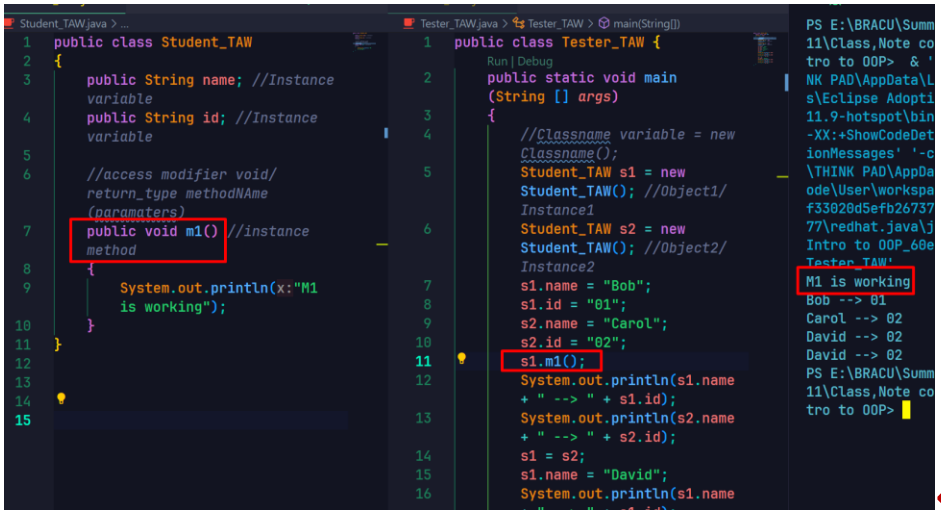



```java
public class Student_TAW
{
    public String name; //
    Instance variable
    public String id; //
    Instance variable
}
```

```java
public class Tester_TAW {
    public static void main (String [] ar
        //Classname variable = new
        Classname();
        Student_TAW s1 = new Student_TAW
        (); //Object1/Instance1
        Student_TAW s2 = new Student_TAW
        (); //Object1/Instance2
        s1.name = "Bob";
        s1.id = "01";
        s2.name = "Carol";
        s2.id = "02";
        System.out.println(s1.name + "
        --> " + s1.id);
        System.out.println(s2.name + "
        --> " + s2.id);
        s1 = s2;
        s1.name = "David";
        System.out.println(s1.name + "
        --> " + s1.id);
        System.out.println(s2.name + "
        --> " + s2.id);
    }
}
```

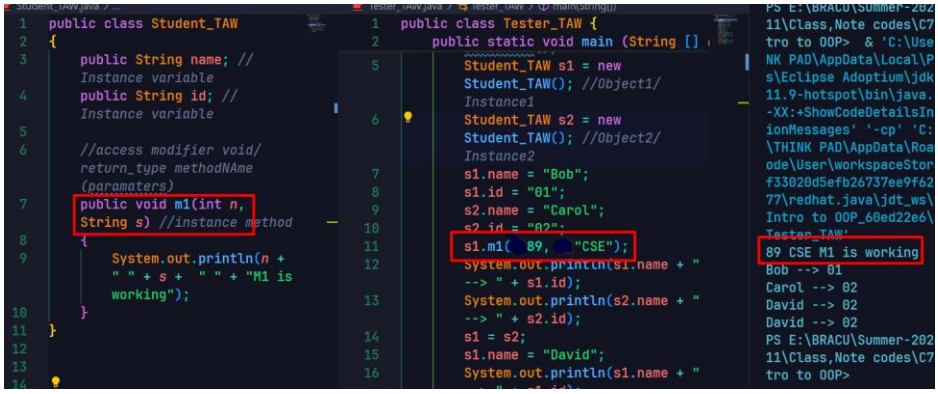
```
Bob --> 01
Carol --> 02
David --> 02
David --> 02
PS E:\BRACU\S
lass,Note codes
OP>
```

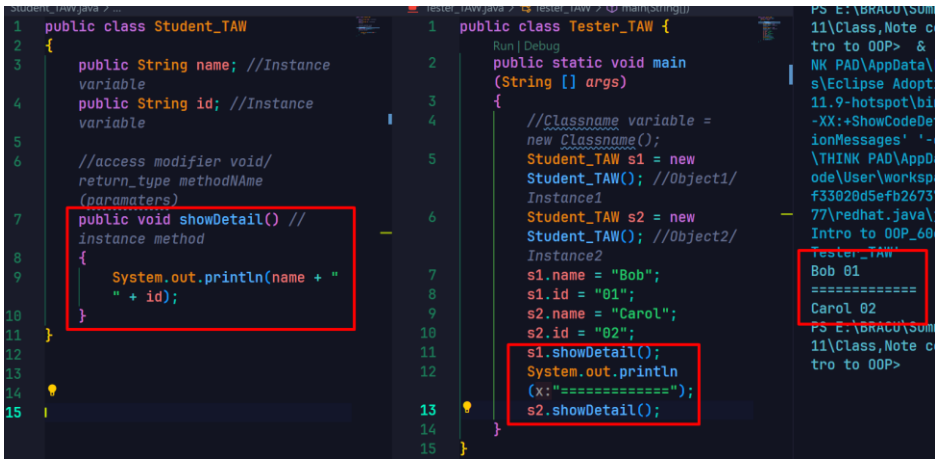

s1 = Dhaka
CTG
Dhaka
name
ID
Bob
11
CTG
s2 = CTG
CTG
name
ID
33
David
s1 = s2

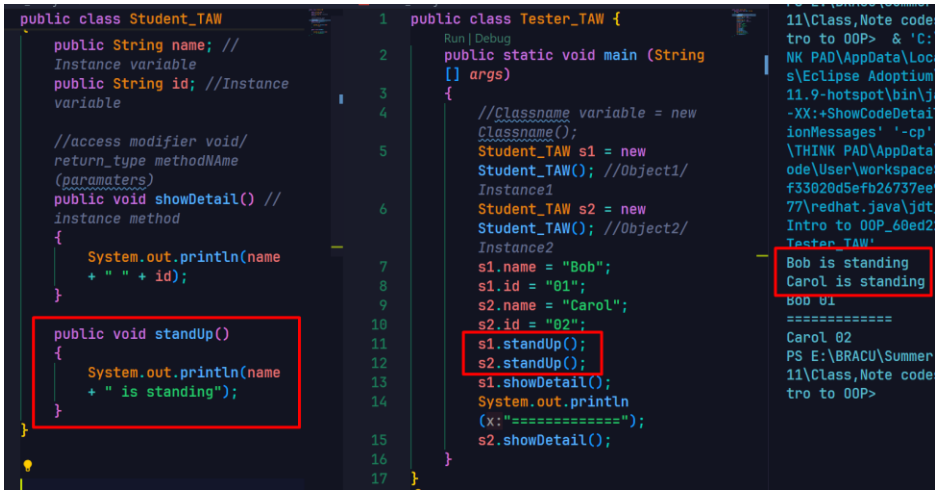


Instance

Method কে access করতে হলে Object এর Reference দরকার হয়।







```
public class Student_TAW
{
    public String name; //
    Instance variable
    public String id; //Instance
    variable

    //access modifier void/
    return_type methodNAme
    (paramaters)
    public void showDetail() //
    instance method
    {
        System.out.println(name
        + " " + id);
    }

    public void standUp()
    {
        System.out.println(name
        + " is standing");
    }
}
```

```
public class Tester_TAW {
    Run | Debug
    public static void main (String
    [] args)
    {
        //Classname variable = new
        Classname();
        Student_TAW s1 = new
        Student_TAW(); //Object1/
        Instance1
        Student_TAW s2 = new
        Student_TAW(); //Object2/
        Instance2
        s1.name = "Bob";
        s1.id = "01";
        s2.name = "Carol";
        s2.id = "02";
        s1.standUp();
        s2.standUp();
        s1.showDetail();
        System.out.println
        (x:"=============");
        s2.showDetail();
    }
}
```

```
11\Class,Note code
tro to OOP>  & 'C:
NK PAD\AppData\Loc
s\Eclipse Adoptium
11.9-hotspot\bin\j
-XX:+ShowCodeDetai
ionMessages' '-cp'
\THINK PAD\AppData
ode\User\workspace
f33020d5efb26737ee
77\redhat.java\jdt
Intro to OOP_60ed2
Tester_TAW'
Bob is standing
Carol is standing
Bob 01
=============
Carol 02
PS E:\BRACU\Summer
11\Class,Note code
tro to OOP>
```



```
House_TAW.java > ...
public class House_TAW {
    public int window;
    public int door;

    public void view()
    {
        System.out.println
        (window + " windows");
        System.out.println(door
        + " doors");
    }
}
```

```
HouseTester_TAW.java > ...
public class HouseTester_TAW {
    Run | Debug
    public static void main (String
    [] args)
    {
        House_TAW h1 = new House_TAW
        ();
        h1.window = 6;
        h1.door = 2;
        House_TAW h2 = new House_TAW
        ();
        h2.window = 4;
        h2.door = 2;
        System.out.println(x:"h1
        =========");
        h1.view();
        System.out.println(x:"h2
        =========");
        h2.view();
    }
}
```

```
PS E:\BRACU\Sum
11\Class,Note c
tro to OOP>  &
NK PAD\AppData\
s\Eclipse Adopt
11.9-hotspot\bi
-XX:+ShowCodeDe
ionMessages' '-
\THINK PAD\AppD
ode\User\worksp
f33020d5efb2673
77\redhat.java\
Intro to OOP_60
HouseTester_TAW
h1 =========
6 windows
2 doors
h2 =========
4 windows
2 doors
PS E:\BRACU\Sum
11\Class,Note c
tro to OOP>
```

House_TAW.java HouseTester_TAW.java

HouseTester_TAW.java > ...

```java
public class HouseTester_TAW {
    Run | Debug
    public static void main (String [] args)
    {
        House_TAW h1 = new House_TAW();
        h1.window = 6;
        h1.door = 2;
        House_TAW h2 = new House_TAW();
        h2.window = 4;
        h2.door = 2;
        System.out.println(x:"h1 =========");
        h1.view();
        System.out.println(x:"h2 =========");
        h2.view();
        h1.increaseDoor(dr:1);
        System.out.println(x:"h1 =========");
        h1.view();
        h2.increaseDoor(dr:2);
        System.out.println(x:"h2 =========");
        h2.view();
    }
}
```

TERMINAL

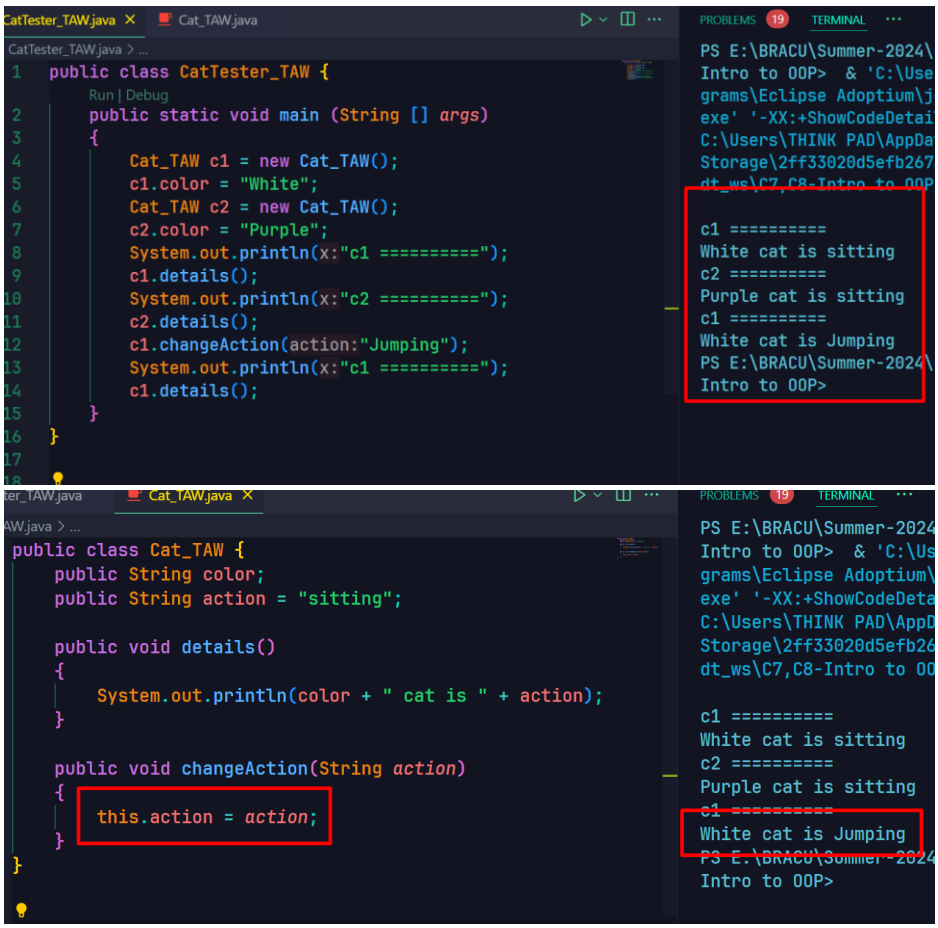
```
tro to OOP>  &
ograms\Eclipse A
ava.exe' '-XX:+S
'-cp' 'C:\Users\
\workspaceStorag
redhat.java\jdt_
 'HouseTester_TA
h1 =========
6 windows
2 doors
h2 =========
4 windows
2 doors
h1 =========
6 windows
3 doors
h2 =========
4 windows
4 doors
PS E:\BRACU\Summ
C8-Intro to OOP>
```

ouse_TAW.java > ...

```java
public class House_TAW {
    public int window;
    public int door;

    public void view()
    {
        System.out.println(window + " windows");
        System.out.println(door + " doors");
    }

    public void increaseDoor(int dr)
    {
        door += dr;
    }
}
```



```
CatTester_TAW.java
public class CatTester_TAW {
    Run | Debug
    public static void main (String [] args)
    {
        Cat_TAW c1 = new Cat_TAW();
        c1.color = "White";
        Cat_TAW c2 = new Cat_TAW();
        c2.color = "Purple";
        System.out.println(x:"c1 ==========");
        c1.details();
        System.out.println(x:"c2 ==========");
        c2.details();
        c1.changeAction(action:"Jumping");
        System.out.println(x:"c1 ==========");
        c1.details();
    }
}
```

```
PS E:\BRACU\Summer-2024\
Intro to OOP>  & 'C:\Use
grams\Eclipse Adoptium\j
exe' '-XX:+ShowCodeDetai
C:\Users\THINK PAD\AppDa
Storage\2ff33020d5efb267
dt_ws\C7,C8-Intro to OOP

c1 ==========
White cat is sitting
c2 ==========
Purple cat is sitting
c1 ==========
White cat is Jumping
PS E:\BRACU\Summer-2024\
Intro to OOP>
```



```
Cat_TAW.java
public class Cat_TAW {
    public String color;
    public String action = "sitting";

    public void details()
    {
        System.out.println(color + " cat is " + action);
    }

    public void changeAction(String action)
    {
        this.action = action;
    }
}
```

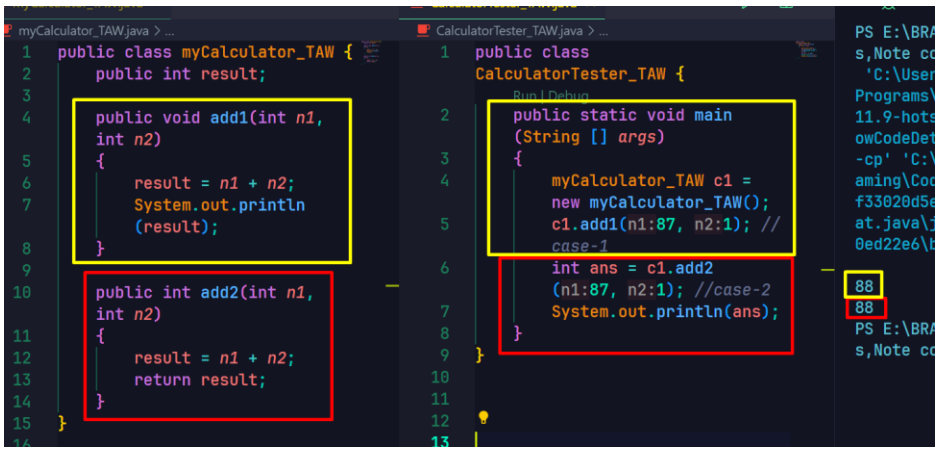
```
PS E:\BRACU\Summer-2024
Intro to OOP>  & 'C:\Us
grams\Eclipse Adoptium\
exe' '-XX:+ShowCodeDeta
C:\Users\THINK PAD\AppD
Storage\2ff33020d5efb26
dt_ws\C7,C8-Intro to OO

c1 ==========
White cat is sitting
c2 ==========
Purple cat is sitting
c1 ==========
White cat is Jumping
PS E:\BRACU\Summer-2024
Intro to OOP>
```

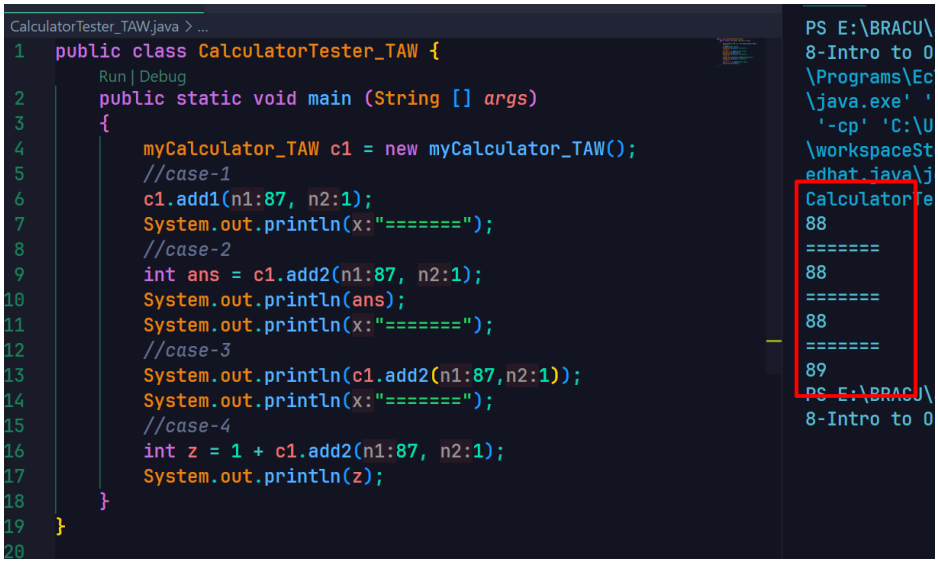
Print statement থাকলে return করবে। যেমনঃ

System.out.println(s1.details());
s1.details(); এইক্ষেত্রে return type হবে void.

OOP Basic Practice Problems



অর্থাৎ, void থাকলে method এর ভিতরে print statement [System.out.println()] থাকবে। আর Datatype [int, String.....etc] উল্লেখ থাকলে return করবে।



CalculatorTester_TAW.java myCalculator_TAW.java

myCalculator_TAW.java > ...

```java
public class myCalculator_TAW {
    public int result;

    public void add1(int n1, int n2)
    {
        result = n1 + n2;
        System.out.println(result);
    }

    public int add2(int n1, int n2)
    {
        result = n1 + n2;
        return result;
    }
}
```

Method কে ৩ ভাবে call করা যায়।

(i) a1.methodA();
(ii) int b = a1.methodA();
(iii) System.out.println(a1.methodA());

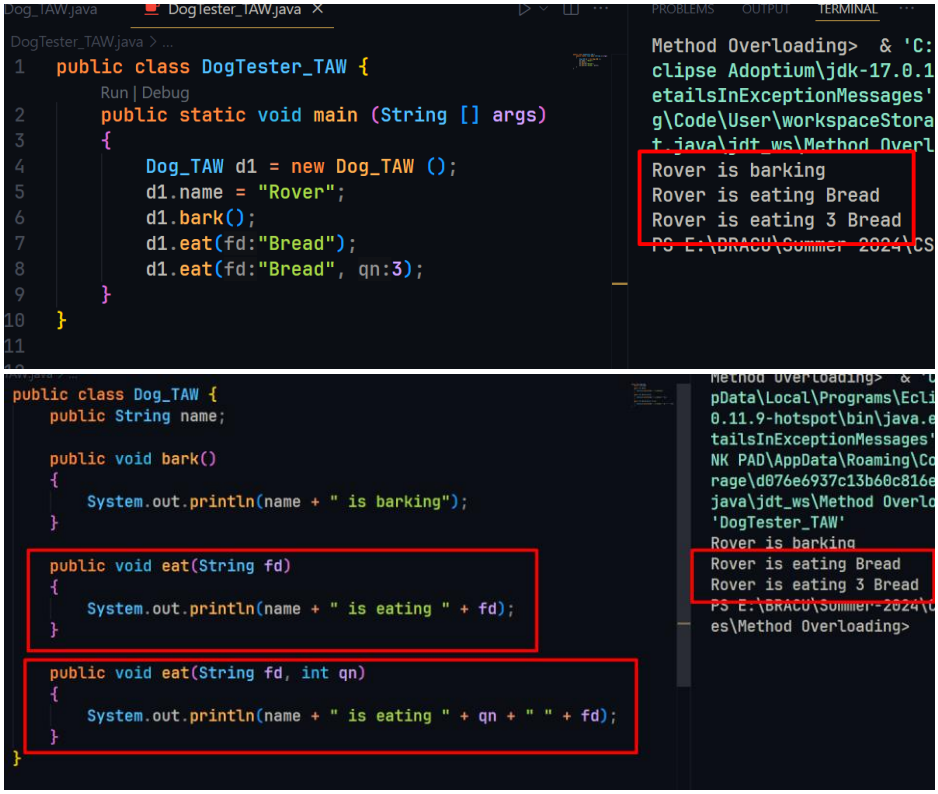
Method Overloading



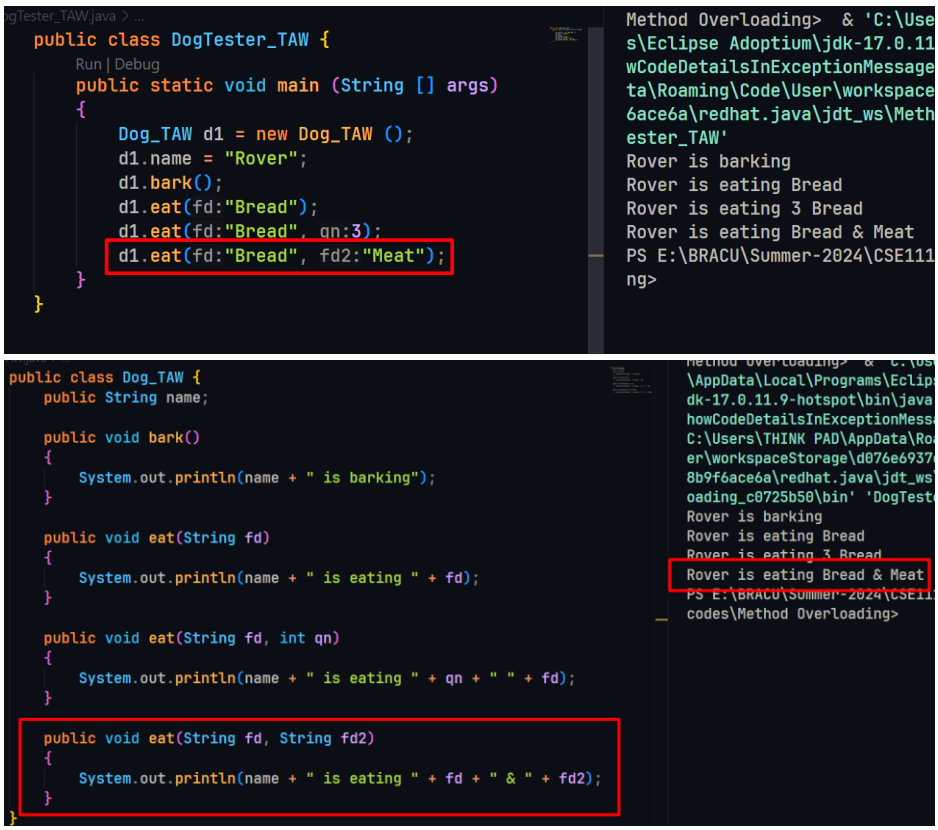
এখানে, ১ম টা কাজ করছে কিন্তু ২য় টা কাজ করছে না।



এক্ষেত্রে, ২য় টা কাজ করছে কিন্তু ১ম টা কাজ করছে না।



একই class এ একই নামে যদি একাধিক method থাকে বিভিন্ন ধরণের parameter [String fd, int qn] এর সাথে তখন তাকে আমরা বলবো Method Overloading.



```
public class DogTester_TAW {
    Run | Debug
    public static void main (String [] args)
    {
        Dog_TAW d1 = new Dog_TAW ();
        d1.name = "Rover";
        d1.bark();
        d1.eat(fd:"Bread");
        d1.eat(fd:"Bread", qn:3);
        d1.eat(fd:"Bread", fd2:"Meat");
    }
}
```

```
Method Overloading>  & 'C:\Use
s\Eclipse Adoptium\jdk-17.0.11
wCodeDetailsInExceptionMessage
ta\Roaming\Code\User\workspace
6ace6a\redhat.java\jdt_ws\Meth
ester_TAW'
Rover is barking
Rover is eating Bread
Rover is eating 3 Bread
Rover is eating Bread & Meat
PS E:\BRACU\Summer-2024\CSE111
ng>
```

```
public class Dog_TAW {
    public String name;

    public void bark()
    {
        System.out.println(name + " is barking");
    }

    public void eat(String fd)
    {
        System.out.println(name + " is eating " + fd);
    }

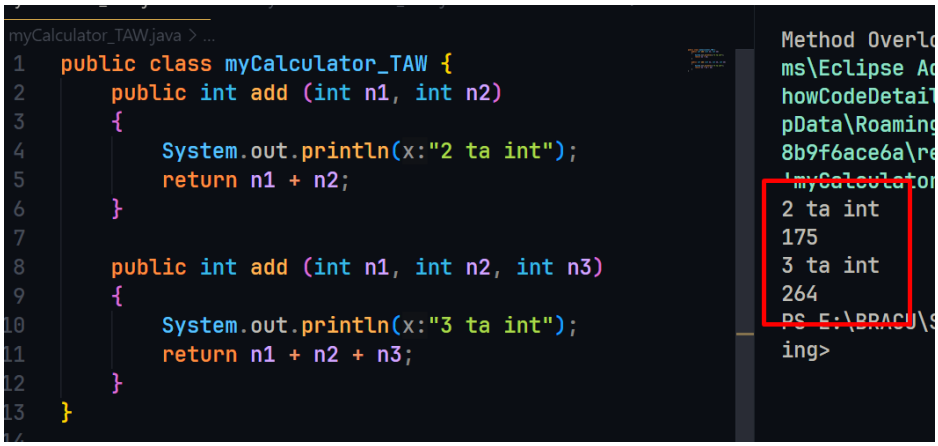
    public void eat(String fd, int qn)
    {
        System.out.println(name + " is eating " + qn + " " + fd);
    }

    public void eat(String fd, String fd2)
    {
        System.out.println(name + " is eating " + fd + " & " + fd2);
    }
}
```

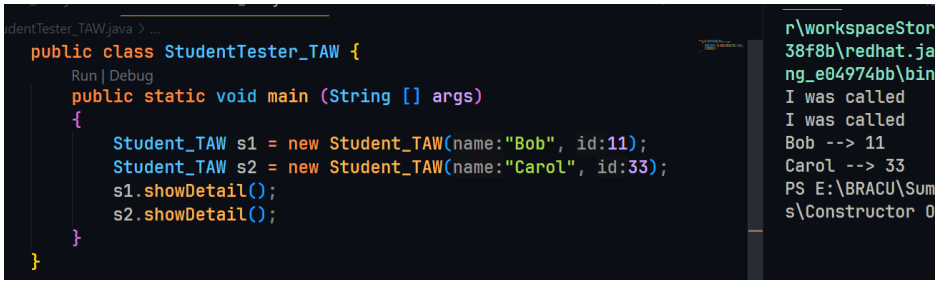
```
\AppData\Local\Programs\Eclip
dk-17.0.11.9-hotspot\bin\java
howCodeDetailsInExceptionMess
C:\Users\THINK PAD\AppData\Ro
er\workspaceStorage\d076e6937
8b9f6ace6a\redhat.java\jdt_ws
oading_c0725b50\bin' 'DogTest
Rover is barking
Rover is eating Bread
Rover is eating 3 Bread
Rover is eating Bread & Meat
codes\Method Overloading>
```

এখানে ক্রম বা Order ক্রমান্বয়ে সাজানো জরুরি না। আগে বা পরে এক জায়গায় থাকলেই হবে।


Calculator_TAW.java
myCalculatorTester_TAW.java
CalculatorTester_TAW.java > ...
public class myCalculatorTester_TAW {
Run | Debug
public static void main (String [] args)
{
myCalculator_TAW c1 = new myCalculator_TAW();
System.out.println(c1.add(n1:87, n2:88));
System.out.println(c1.add(n1:87, n2:88, n3:89));
}
}
TERMINAL
Method Over
l\Programs\
ava.exe' '-
' 'C:\Users
ceStorage\d
jdt_ws\Meth
ster_TAW'
2 ta int
175
3 ta int
264
PS E:\BRACU

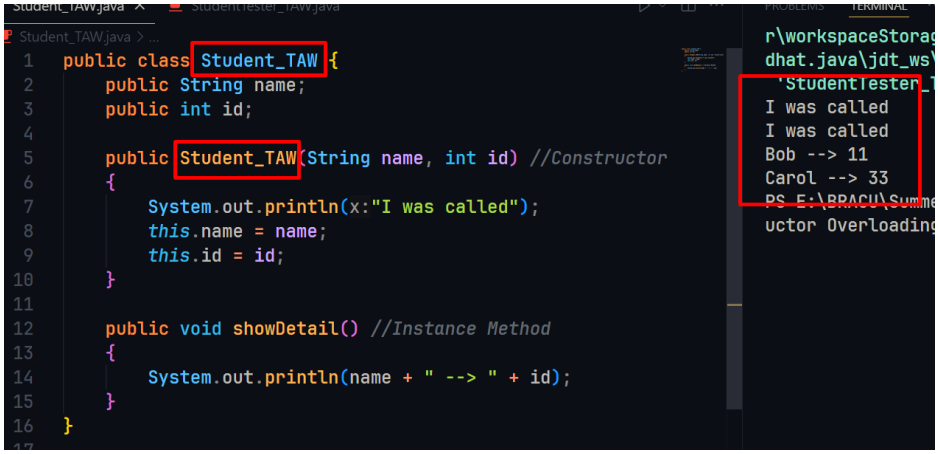

myCalculator_TAW.java > ...
1 public class myCalculator_TAW {
2 public int add (int n1, int n2)
3 {
4 System.out.println(x:"2 ta int");
5 return n1 + n2;
6 }
7
8 public int add (int n1, int n2, int n3)
9 {
10 System.out.println(x:"3 ta int");
11 return n1 + n2 + n3;
12 }
13 }
Method Overlo
ms\Eclipse Ad
howCodeDetail
pData\Roaming
8b9f6ace6a\re
2 ta int
175
3 ta int
264
ing>

Constructor



```java
public class StudentTester_TAW {
    Run | Debug
    public static void main (String [] args)
    {
        Student_TAW s1 = new Student_TAW(name:"Bob", id:11);
        Student_TAW s2 = new Student_TAW(name:"Carol", id:33);
        s1.showDetail();
        s2.showDetail();
    }
}
```

```
r\workspaceStor
38f8b\redhat.ja
ng_e04974bb\bin
I was called
I was called
Bob --> 11
Carol --> 33
PS E:\BRACU\Sum
s\Constructor O
```



```java
public class Student_TAW {
    public String name;
    public int id;

    public Student_TAW(String name, int id) //Constructor
    {
        System.out.println(x:"I was called");
        this.name = name;
        this.id = id;
    }

    public void showDetail() //Instance Method
    {
        System.out.println(name + " --> " + id);
    }
}
```

```
r\workspaceStorag
dhat.java\jdt_ws\
'StudentTester_1
I was called
I was called
Bob --> 11
Carol --> 33
PS E:\BRACU\Summe
uctor Overloading
```

Class Name & Constructor Name একই হতে হবে।